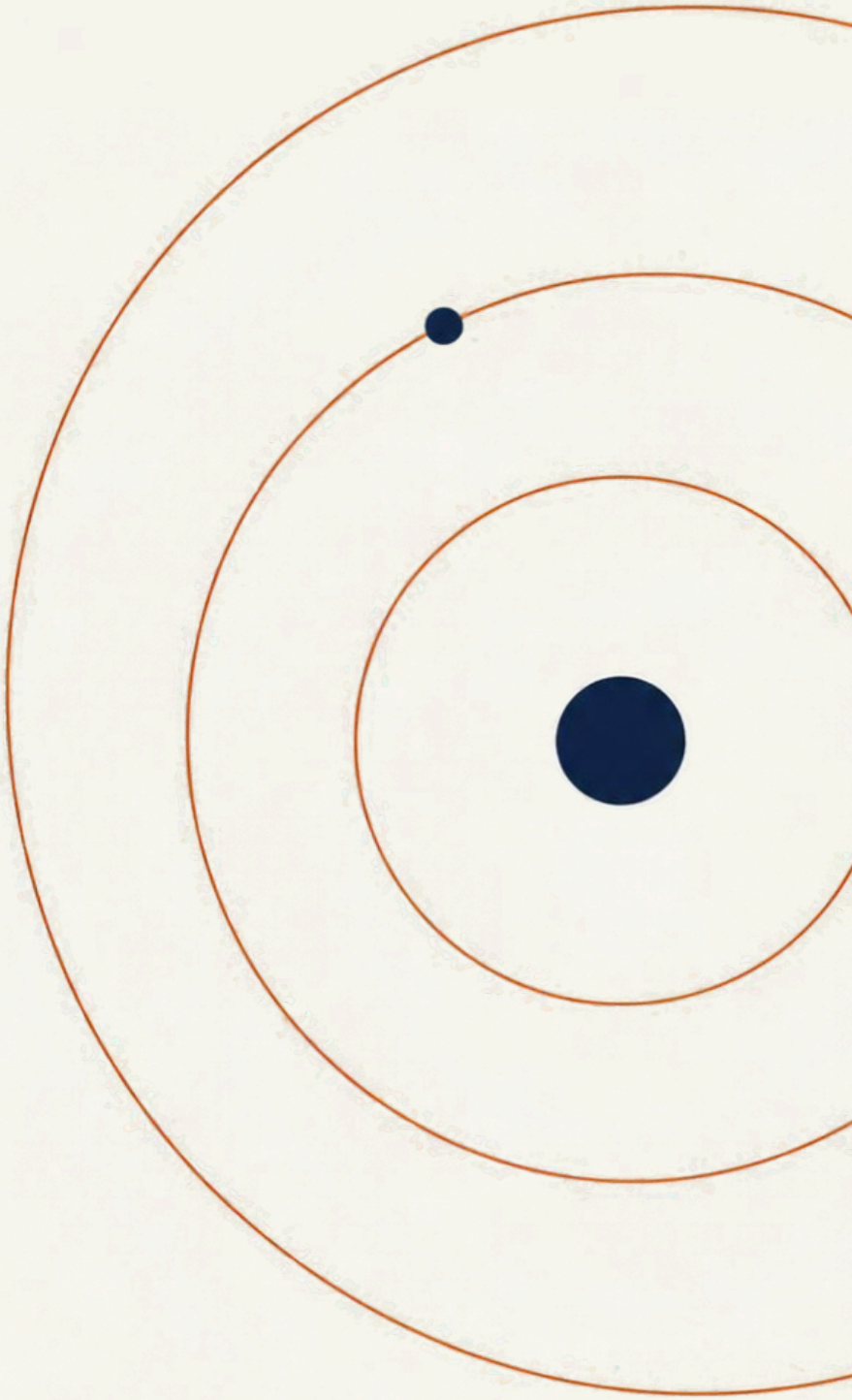
Exploring Gravitational Microlensing

From Physics to Machine Learning

A computational framework for generating synthetic light curves and identifying planetary signatures using machine learning



Roc Rubió Cardona
Treball de Recerca
Tutor: Jordi Roselló, Núria Peris
Class 25-27
INS Alfons Costafreda
2n Batxillerat B

Acknowledgements

I would like to express my sincere gratitude to my tutors for their guidance throughout this project. In particular, to Jordi Roselló, whose constant support, revisions, and dedication were essential from the very beginning and throughout the majority of this work. Equally, I would like to thank Núria Peris for her valuable guidance during the final stages of the project.

I also wish to extend a special acknowledgement to Eduardo Bendek, researcher at NASA, whose scientific advice and encouragement provided both direction and motivation during the development of this work.

Abstract

English

Gravitational microlensing is one of the few methods capable of detecting exoplanets at large orbital separations, yet its underlying physics, rooted in general relativity and complex lens geometry, remains largely inaccessible outside of specialized academic research.

This work presents a complete, end-to-end study of the phenomenon: from the derivation and implementation of the single-lens and binary-lens equations, through the construction of a synthetic dataset generator calibrated with empirical OGLE-IV photometric noise, cadence gaps, and blending, to the training and evaluation of machine learning classifiers for automated single-vs-binary event detection. Two convolutional neural networks and a gradient-boosted tree were trained on synthetic data, independently converging at an AUC of approximately 0.60 on realistic curves (a ceiling shown to be a physical property of the data, not a limitation of the models). When evaluated against 92 real, confirmed, planetary microlensing events from the OGLE-IV survey, the CNN successfully flagged 53.3% of them, demonstrating that a model trained entirely on synthetic data generalises to genuine astronomical observations.

The entire pipeline (dataset generation, statistical validation, and model inference) is integrated into an interactive web application designed to make this complex astrophysical topic as accessible and comprehensible as possible. All code, models, and data are publicly available at https://github.com/rocrubio/Microlensing-Tool-Hub

Keywords: Gravitational Microlensing, Exoplanet Detection, Light Curve Simulation, Machine Learning, OGLE-IV, Binary-lens Systems.

Catalan

La Microlent Gravitatòria és un dels pocs mètodes que permeten de detectar exoplanetes a grans separacions orbitals; no obstant, la seva física subjacent, arrelada en la relativitat general i la geometria complexa de les lents, continua sent en gran part inaccessible fora de la recerca acadèmica especialitzada.

Aquest treball presenta un estudi complet del fenomen: des de la derivació i implementació de les equacions de lent simple i lent binària, passant per la construcció d'un generador de conjunts de dades sintètics calibrat amb soroll fotomètric empíric extret d'OGLE-IV, buits de cadència i barreja lumínica, fins a l'entrenament i avaluació de classificadors d'aprenentatge automàtic per a la detecció automatitzada d'esdeveniments simples o binaris. S'han entrenat dues xarxes neuronals convolucionals i un conjunt d'arbres de decisió potenciats per gradient amb amb dades sintètiques, convergint independentment a una AUC d'aproximadament 0,60 en corbes teòricament reals, un sostre que s'ha demostrat que és una propietat física de les dades, no una limitació dels models. En ser avaluada amb 92 esdeveniments de microlent binària reals i confirmats del sondeig OGLE-IV, la xarxa neuronal convolucional ha identificat correctament el 53,3% d'aquests, demostrant que un model entrenat completament amb dades sintètiques generalitza a observacions astronòmiques genuïnes.
Tot el procés (generació de conjunts de dades, validació estadística i inferència del model) està integrat en una aplicació web interactiva dissenyada per fer que aquest tema astrofísic complex sigui el més accessible i comprensible possible.
Tot el codi, els models i totes les dades són públiques i accessibles des de https://github.com/rocrubio/Microlensing-Tool-Hub

Paraules clau: Microlent gravitatòria, Detecció d'exoplanetes, Simulació de corbes de llum, Aprenentatge automàtic, OGLE-IV, Sistemes de lent binària.

Castellano

La Microlente Gravitatoria es uno de los pocos métodos que permiten detectar exoplanetas a grandes separaciones orbitales; no obstante, su física subyacente, arraigada en la relatividad general y la geometría compleja de las lentes, sigue siendo en gran parte inaccesible fuera de la investigación académica especializada.
Este trabajo presenta un estudio completo del fenómeno: desde la derivación e implementación de las ecuaciones de lente simple y lente binaria, pasando por la construcción de un generador de conjuntos de datos sintéticos calibrado con ruido fotométrico empírico extraído de OGLE-IV, vacíos de cadencia y mezcla lumínica, hasta el entrenamiento y evaluación de clasificadores de aprendizaje automático para la detección automatizada de eventos simples o binarios. Se han entrenado dos redes neuronales convolucionales y un conjunto de árboles de decisión potenciados por gradiente con datos sintéticos, convergiendo independientemente a una AUC

de aproximadamente 0,60 en curvas teóricamente reales, un techo que se ha demostrado que es una propiedad física de los datos, no una limitación de los modelos. Al ser evaluada frente a 92 eventos de microlente binaria reales y confirmados del sondeo OGLE-IV, la red neuronal convolucional ha identificado correctamente el 53,3% de los mismos, demostrando que un modelo entrenado completamente con datos sintéticos generaliza a observaciones astronómicas genuinas.

Todo el proceso (generación de conjuntos de datos, validación estadística e inferencia del modelo) está integrado en una aplicación web interactiva diseñada para hacer que este tema astrofísico complejo sea lo más accesible y comprensible posible.

Todo el código, los modelos y todos los datos son públicos y accesibles en https://github.com/rocrubio/Microlensing-Tool-Hub

Palabras clave: Microlente gravitatoria, Detección de exoplanetas, Simulación de curvas de luz, Aprendizaje Automático, OGLE-IV, Sistemas de lente binaria.

Table of contents

Introduction....9

Exoplanet detection methods....12

Geometry of a microlensing system....14

Single lens equations....15

- Einstein’s radius....15
- Einstein’s time....15
- Normalized lens - source separation by time....16
- Paczyński’s amplification....16

Binary lens equations....18

- Masses ratio....18
- Einstein radius....18
- System’s Einstein angle....19
- Einstein’s time....20
- Semi-major axis....21
- Angular velocity....21
- Orbital angle....22
- Orbital time....22
- Planet-lens distance (normalized)....22
- Planet’s maximum impact time....23
- Trajectory angle by time....24
- Lens position....24
- Source position....24
- Planet position....25
- Jacobian....25

Parameter distributions....27

- Lens’ Mass (M_⋆)....27
- Mass ratio (q)....28

Distance from lens to source (s)..... 28
Distance to lens (D_l)..... 29
Lens' velocity (v_⊥)..... 31
Impact parameter (u0)..... 31
Semi-major axis (a)..... 32
Orbital eccentricity (e)..... 33
Trajectory angle of reference (αref)..... 34
Source baseline magnitude (I_s)..... 35
Observational imperfections..... 38
OGLE-IV..... 38
Data source..... 38
Photometric noise..... 39
Observing cadence..... 41
Blending..... 43
Machine Learning..... 47
What machine learning is..... 47
The three components of machine learning..... 48
The generalization process..... 48
Types of supervised learning models..... 49
Model..... 51
Task..... 51
Type..... 52
Data used..... 54
Input representation..... 55
Channels..... 56
Simple model..... 56
Real model..... 56
Feature Engineering for the Gradient-Boosted Tree..... 57
Architecture..... 58

Training....61
Results....62
Simple CNN....62
Real CNN....63
GBT....64
Web Application & Interactive Tool Hub....66
Website operation....66
Datasets generation....66
Dataset validation....68
Distribution logic....70
Models....70
Real data....71
Data preprocessing....71
Results....72
Conclusions....73
References....75

Introduction

The search for planets beyond our solar system is one of the most active areas of modern astrophysics. To date, over 6,000 exoplanets have been confirmed [27] using a variety of observational techniques, each sensitive to different planetary and stellar configurations. The discovery of these worlds is not only scientifically significant, as it reshapes our understanding of how planetary systems form and how common they are across the galaxy, but also carries a broader human relevance, as the question of whether other planets capable of supporting life exist remains one of the most compelling open questions in science.

My interest in this topic comes from a long-standing fascination with the universe and with the idea of finding planets beyond our own solar system. Among the available detection methods, gravitational microlensing was the one that caught my attention the most. Its underlying physics (the bending of light by gravity and the way an entire planetary system can be characterized from a single brightness variation) seemed more fascinating than any other approach, and that was reason enough to choose it. The decision to combine microlensing with machine learning was driven by two reasons: a genuine desire to understand how machine learning works and what it can achieve, and a practical one as the volume of microlensing data produced by current and future surveys is too large to be analyzed manually, and automating this process is not only useful but necessary.

According to Einstein's general theory of relativity, gravity curves spacetime and therefore deflects the path of light. When a massive stellar object passes between a distant source and an observer, it acts as a gravitational lens, temporarily amplifying the observed brightness of the background source. Microlensing is a subcategory of this phenomenon where the source and lens are separated by an angular distance on the order of milli-arcseconds [42], and all physical parameters of the event are derived exclusively from the photon intensity variation of the source over time.

Gravitational lensing was first suggested under Newton's corpuscular theory of light, and later its predicted deflection angle was doubled by Einstein's general theory of relativity. Arthur Eddington confirmed this experimentally in 1919 during his solar eclipse expedition. Although Einstein himself stated in 1936 that "there is no great chance of observing this phenomenon" [6], Bohdan Paczyński revived gravitational microlensing in 1986 as an observable phenomenon in

the Milky Way [34]. Initially proposed as a tool to detect dark matter concentrations, this objective was later discarded as dark matter is now known to consist of non-baryonic particles which need not be compact [21]. Today, microlensing is primarily used to detect exoplanets, though its observational difficulty is reflected in the numbers: of the 6,000+ confirmed exoplanets, only approximately 250 have been discovered via microlensing [42].

The objective of this work is to develop a complete machine learning pipeline for the automated analysis of microlensing light curves, capable of classifying the system type on each event. The specific objectives are the following. The first one is to understand where microlensing light curves come from and how the equations that describe them work, and explain them intuitively for the reader to make the hard work of research easier for whoever comes next. The second one is to find the distributions of the parameters that define the events and create a synthetic dataset generator to analyze curves as real as possible. The third one is to develop a machine learning classifier that determines, from a lightcurve alone, whether a microlensing event involves a single lens or a binary lens system. Finally a web will be created trying to join most of the theory, the dataset generator, and the option to feed the machine learning model with real or generated data to get results.

A central methodological challenge in this work is the scarcity of real labeled microlensing data. Microlensing events are non-repeatable (once the lens moves out of alignment, the event is gone forever), and confirmed detections of planetary systems via microlensing remain rare. This makes it impossible to train machine learning models directly on real observations. For this reason, all datasets used in this work are synthetically generated using parameter distributions derived from real observed events and published literature. Before being used for data generation, each parameter distribution is validated against real observational data: stellar mass and orbital parameter distributions are cross-checked against the NASA Exoplanet Science Institute archive [25][26], and impact parameter distributions are validated against the public OGLE-IV catalogue [30]. This ensures that the synthetic light curves are physically realistic even being not directly observed.

The methodology begins with the mathematical derivation of the microlensing equations for both single-lens and binary-lens systems. From these, the parameter distributions governing real microlensing events are characterized and validated against the observational data described above, ensuring that the synthetic datasets used to train the models are physically realistic. Then models will be trained and the web will be crafted and published together with this paper to github for public use and reference.

Note that all code used for dataset generation and model training was written in Python, AI-assisted and subsequently revised and validated by the author.

Exoplanet detection methods

Exoplanets cannot in most cases be observed directly, as they are far too faint and too close to their host stars to be resolved from Earth. Instead, each detection method exploits a different indirect physical effect, making the various techniques complementary: each is sensitive to a different region of the planet mass versus orbital separation parameter space [42].

Radial velocity detects planets by measuring the periodic Doppler shift in the host star's spectrum induced by the gravitational pull of an orbiting planet. It is particularly effective at finding massive planets in short-period orbits, but detecting planets at orbital distances greater than 1 AU requires observing campaigns spanning years to decades, making it poorly suited to the discovery of cold, distant planets [42].

Transit photometry identifies exoplanets by detecting the periodic dimming of a star as a planet passes across its disk. It requires a precise geometric alignment between the planet's orbital plane and the line of sight, so it is inherently biased towards planets orbiting close to their host stars. It is the most prolific detection method to date, with the Kepler space mission alone having found hundreds of confirmed planets [42].

Astrometry measures extremely small periodic shifts in a star's position on the sky caused by the gravitational influence of an orbiting planet. The technique is in principle sensitive to a wide range of orbital separations, but the required positional precision is extremely demanding, and the method has only recently become viable at scale with the Gaia space mission [42].

Pulsar timing exploits the extraordinary regularity of millisecond pulsars (rapidly rotating neutron stars that emit highly stable electromagnetic pulses) to detect planets through the irregularities they introduce in pulse arrival times. It provided the first confirmed exoplanet detections in 1992, but is restricted to an exotic and rare class of stellar remnants [42].

Direct imaging is the only method that attempts to capture light emitted or reflected by the planet itself. Using adaptive optics on large telescopes, it has successfully found massive planets at very large separations from their host stars. However, because a star outshines any nearby planet by many orders of magnitude in the optical, only planets lying far from their hosts in young systems are detectable with this method [42].

Gravitational microlensing is unique among detection methods in that it requires no light to be detected from either the planet or its host star. It exploits the temporary brightening of a background source star as a foreground lens system passes in front of it, with the planet revealing itself through a brief anomaly superimposed on the main lensing event. This makes it sensitive to planets at orbital separations of roughly 1–10 AU around stars at kiloparsec distances (a regime largely inaccessible to all other methods) and it does not favor any particular type of host star [42]. Its main limitation is that events are non-repeatable and require dedicated wide-field survey programs to detect at scale.

Geometry of a microlensing system

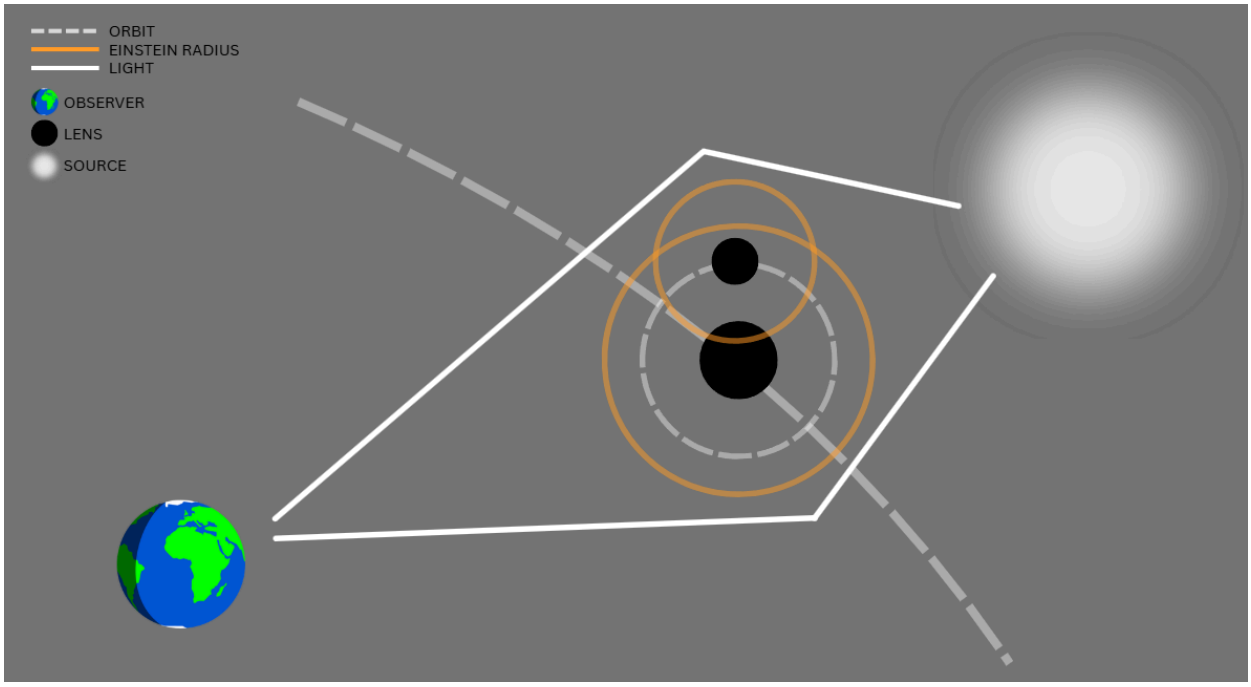


Image 1: Understanding a microlensing system's geometry. Source: Personal elaboration

**The proportions and distances shown in the image may not be probable in reality*

A microlensing system consists of three elements: an observer (typically a telescope), a distant light-emitting source, and at least one intervening mass known as the lens, which curves spacetime and deflects the incoming light [21][31].

In practice, although all components are in motion, microlensing analyzes typically assume a fixed alignment between the source and the observer while the lens transits the line of sight between them.

The observed intensity increases because the gravitational curvature deflects not only the light ray that would ordinarily reach the telescope, but also neighbouring rays, so the observer receives a higher photon flux than in the absence of the lens.

Single lens equations

The single lens equations yield a plot of photon intensity as a function of time known as the light curve, which quantifies the amplification of the source flux at each moment. The final result is Paczyński's amplification, a compact expression that nonetheless requires several intermediate quantities to be derived beforehand.

Einstein’s radius

Einstein’s radius is the first equation needed and it will be the most used also in the binary and N lenses equations as it represents the influence of the lens on the light deflection.

$$r_E = \sqrt{\frac{4GM}{c^2}\ \frac{D_l D_{ls}}{D_s}}$$

r_E	Radius where gravity bends light	m
G	Universal Gravitational Constant	$m^3\ kg^{-1}\ s^{-2}$
M	Lens’ mass	kg
c	Light’s velocity	m/s
D_{ls}	Lens - source distance	m
D_l	Lens - observer distance	m
D_s	Source - observer distance	m

Einstein’s time

Einstein’s time equation provides the time needed by the lens to complete 1 Einstein radius

$$t_E = \frac{r_E}{v_\perp}$$

t_E	Time for the lens to cross one Einstein radius	s
r_E	Einstein's Radius	m
$v_\perp$	Lens' velocity	m/s

Normalized lens - source separation by time

The normalized separation gives the instantaneous lens-source angular separation expressed in units of the Einstein radius.

$$u(t) = \sqrt{u_0^2 + \left(\frac{t - t_0}{t_E}\right)^2}$$

u(t)	Lens - source separation normalized at each moment	–
u_0	Normalized minimum separation distance	–
t	Time (in the event) where it has to be calculated	s
t_0	Second during the event with maximum alignment	s
t_E	Einstein's time	s

Paczyński's amplification

Paczyński's amplification is the final expression, giving the instantaneous magnification factor of the source flux as a function of the normalized separation

$$A(u) = \frac{u^2 + 2}{u\sqrt{u^2 + 4}}$$

A(u)	Photon intensity increase	–
u	Normalized lens - source separation by time	–

Binary lens equations

Everything from the binary lens equations can be extended to N lenses, but as increasing the lens number by more than 2 also increases the difficulty on calculus and probability of being recorded, we are going to focus just on the binary lens equations.

Binary lens equations are also derived under the thin lens approximation. They consider a primary lens, as in the single lens case, plus a secondary lens, typically a planet, orbiting the primary and producing anomalies in the single-lens light curve. [45]

The main conceptual difference is that binary lens equations incorporate the time-varying positions of both masses, whereas the single lens case depends only on mass, distances, and relative velocity.

Masses ratio

The masses ratio, or masses fraction, is a preliminary step that, while seemingly redundant, simplifies the mass-dependent terms in all subsequent equations.

$$m_\star = \frac{M_\star}{M_\star + M_p} \qquad m_p = \frac{M_p}{M_\star + M_p} \qquad q = \frac{m_p}{m_\star}$$

$M_\star$	Main lens mass (star)	kg
M_p	Secondary lens mass (planet)	kg
$m_\star$	Star mass' percentage over the system	–
m_p	Planet mass' percentage over the system	–
q	Masses proportion	–

Einstein radius

We already know the formula for the Einstein radius from the single lens equations, but now, as we have two different lenses, we would have to calculate the radius twice. To reduce the

number of calculations needed we are going to use the mass fractions (q). The planet's Einstein radius is obtained by multiplying the star's Einstein radius by √q. This follows directly from the definition, since the only difference between the two is the mass, and $r_{E(p)} = \sqrt{q} \cdot r_{E\star}$ yields the same result as computing each independently.

$$r_{E_\star} = \sqrt{\frac{4GM_\star}{c^2}\,\frac{D_l D_{ls}}{D_s}} \qquad r_{E_p} = r_{E_\star}\sqrt{q}$$

$r_{E(\star)}$	Radius where lens' gravity bends light	m
$r_{E(p)}$	Radius where planet's' gravity bends light	m
G	Universal Gravitational Constant	$m^3\ kg^{-1}\ s^{-2}$
M	Lens' mass	kg
c	Light's velocity	m/s
D_{ls}	Lens - source distance	m
D_l	Lens - observer distance	m
D_s	Source - observer distance	m
q	Masses fraction	–

System's Einstein angle

System's Einstein Angle represents the angle formed by the lensing system's center to the system's Einstein radius. As the distances involved are enormous, the Einstein angle is typically extremely small so measured in milliarcseconds (mas) instead of radians (rad). Although the Einstein angle can be expressed as $\theta_E = r_E / D_l$, the full expression is used here to make the physical dependencies explicit.

$$\theta_{E_\star} = \sqrt{\frac{4GM_{tot}}{c^2}\,\frac{D_{ls}}{D_l D_s}}$$

θ_E	Angle formed by observer-lens and observer-Einstein radius	rad
G	Universal Gravitational Constant	$m^3\ kg^{-1}\ s^{-2}$
M_{tot}	System's total mass ($M_\star + M_p$)	kg
c	Light's velocity	m/s
D_{ls}	Lens - source distance	m
D_l	Lens - observer distance	m
D_s	Source - observer distance	m

Einstein's time

Einstein's time is the characteristic time for the lens to pass through one Einstein's radius.
We already know this equation from the single lens equation but now, as in Einstein's radius, we have got to implement it also on the planet's mass.
When calculating Einstein's time for the planet the planet's orbital velocity is neglected, and the same transversal velocity as the primary lens is assumed.

$$t_{E_\star} = \frac{r_{E_\star}}{v_\perp} \qquad\qquad t_{E_p} = t_{E_\star}\sqrt{q}$$

$t_{E(\star)}$	Lens' time to complete 1 Einstein radius	s
$t_{E(p)}$	Planet's time to complete 1 Einstein radius	s
$r_{E(\star)}$	Lens' Einstein's radius	m
$v_\perp$	Lens' velocity	m/s
q	Masses fraction	–

Up to here we've defined the formulas for the main part of the binary lens understanding.

The following formulas will help define values for the ones after them which will define position to each component of the system.

Semi-major axis

The semi-major axis defines the size of the orbital ellipse described by the planet around the lens. In the special case of a circular orbit, which is rarely observed, the semi-major axis equals the orbital radius.

$$a = \frac{r_{per} + r_{apt}}{2}$$

a	Semi-major axis	m
r_{per}	Distance from the lens to the periastron of the ellipse	m
r_{apt}	Distance from the lens to the apastron of the ellipse	m

Angular velocity

Angular velocity measures how rapidly the planet orbits, in radians per second.

$$\omega = \sqrt{\frac{GM_{\star}}{a^3}}$$

ω	Planet's angular velocity	rad/s
G	Universal Gravitational Constant	$m^3\ kg^{-1}\ s^{-2}$
$M_{\star}$	Lens' mass	kg
a	Planet's ellipse semi-major axis	m

Orbital angle

The orbital angle, as its name suggests, is the angle between the reference x-axis and the planet's position vector relative to the lens.

$$a(t) = a_0 + \omega\,(t - t_{ref})$$

a(t)	Planet’s orbital angle	rad
a_0	Planet’s initial angle (often defined to 0 or by the lens)	rad
ω	Angular velocity	rad/s
t	Time when the angle is calculated	s
t_{ref}	Time when a_0 = a(t), (often set to 0 also)	s

Orbital time

The orbital time is a quantity that, when directly observable, allows the angular velocity to be derived straightforwardly. In practice, however, it is rarely measured directly from a microlensing event, so the angular velocity is instead computed from the physical parameters and the orbital time is derived from it.

$$T_{orb} = 2\pi/\omega$$

T_{orb}	Time for the planet to complete one lap around the star	s
ω	Planet’s angular velocity	rad/s

Planet-lens distance (normalized)

The planet-lens distance varies over time, as the orbit is generally elliptical. After computing the planet-lens distance, it is normalized by the lens' Einstein radius and used in this dimensionless form in subsequent equations.

$$r(t) = \frac{a\,(1-e^2)}{1 + e\,cos[a(t)]} \qquad d(t) = \frac{r(t)}{r_{E_\star}}$$

d(t)	Normalized planet-lens distance	–
r(t)	Planet-lens distance	m
a	semi-major axis	m
e	Orbital eccentricity (r_{apt} - r_{per} / r_{apt} + r_{per})	–
a(t)	Orbital angle	rad
$r_{E(\star)}$	Lens' Einstein radius	rad

Planet's maximum impact time

This parameter is the analogue of t_0 for the planetary component: it gives the time at which the source passes closest to the planet's Einstein ring, producing the maximum planetary amplification.

$$t_p = t_0 + t_{E_\star} * d\,cos[a]$$

t_p	Planet's maximum impact time	s
t_0	Maximum amplification time	s
$t_{E(\star)}$	Lens' Einstein time	s
d	Planet-lens normalized distance (d(t))	–
a	Orbital Angle (a(t))	rad

Trajectory angle by time

The trajectory angle is the main determinant of how the planet's gravitational influence manifests in the observed light curve. It evolves slowly for planets with long orbital periods relative to the event duration, and remains approximately constant when the orbital time greatly exceeds the Einstein time.

$$\alpha(t) = a_{ref} - a(t)$$

α(t)	Trajectory angle by time	rad
α_{ref}	Trajectory angle at the start of the event	rad
a(t)	Planet's orbital angle	rad

Now we have finished understanding the main equations, and we are going to proceed to the positions which are going to be a fundamental part of the final amplification equation.
First of all, in microlensing, we fix the lens position to (0,0) and therefore, the source and planet are moving.

Lens position

As stated above, the lens is fixed at the origin of the reference frame, (0,0).

$$\vec{\xi}_{\star} = (0, 0)$$

Source position

The source position by the lens will be changing by time.

$$\vec{u}(t) = \left(\left(\frac{t - t_0}{t_{E_\star}}\right), u_0\right)$$

$\vec{u}$	Source position	–
t	The time when the position is calculated	s

t_0	Maximum amplification time	s
$t_{E(\star)}$	Lens Einstein time	s
u_0	Normalized minimum separation distance	–

Planet position

$$\vec{\xi}_p(t) = d(t) * (cos[a(t)], sin[a(t)])$$

$\vec{\xi}_p(t)$	Planet's position by time	–
d(t)	Normalized planet-lens distance	–
a(t)	Orbital angle	rad

Now we have understood all the equations needed to calculate the final amplification which is calculated by a jacobian.

Jacobian

$$J = I - \sum_{i=1}^{2} m_i \frac{\partial}{\partial \vec{u}} \left[\frac{\vec{u} - \vec{\xi}_i}{\left| \vec{u} - \vec{\xi}_i \right|^2} \right]$$

The full derivation of this Jacobian exceeds the scope of this work. The expression is expanded into its component form, although it is not the version implemented computationally.
In this jacobian, *i* represents the mass being currently calculated, so it has to be calculated for the main lens and also the planet.

$$J = I - \sum_{i=1}^{2} m_i \frac{1}{\left| \vec{r}_i \right|^4} \begin{pmatrix} y_i^2 - x_i^2 & -2x_i y_i \\ -2x_i y_i & x_i^2 - y_i^2 \end{pmatrix}$$

**Note that it is not possible to display matrix in google docs, though the terms in brackets are supposed to be a matrix*

The raw amplification A(t) is the invert of the jacobian's determinant so 1/det(J) and it represents how much brighter the source star got at each second (t) over the start

J	The jacobian matrix	–
I	The identity matrix ($^{1}_{0}\,^{0}_{1}$)	–
m_i	i component mass fraction	–
$\vec{r}_i$	$\vec{u} - \vec{\xi}_i$ Source-mass distance	–
x_i	X components from the r vector	–
y_i	Y components from the r vector	–

Parameter distributions

With the microlensing equations established, the next step is to generate the synthetic datasets on which the machine learning models will be trained. For the datasets to produce generalizable models, its parameters must follow distributions consistent with those observed in real events. Even though the number of confirmed microlensing detections is limited, the distributions are derived from real data available in published catalogues and from values reported in the literature, which are described individually in the subsections below.

Lens' Mass (M_⋆)

While all parameters described in this section contribute to the event, the lens mass is particularly central as it directly determines the Einstein radius.

I have extracted the data from the Nasa's exoplanet science institute where I found a table containing almost all values for a microlensing event defined from real events, real binary events. You can find the link to this table at [25].

To plot the distribution of the lens' mass I have used a Python code and the results show a distribution peaking at approximately 0.45 M⊙ and declining steeply beyond 0.8 M⊙.

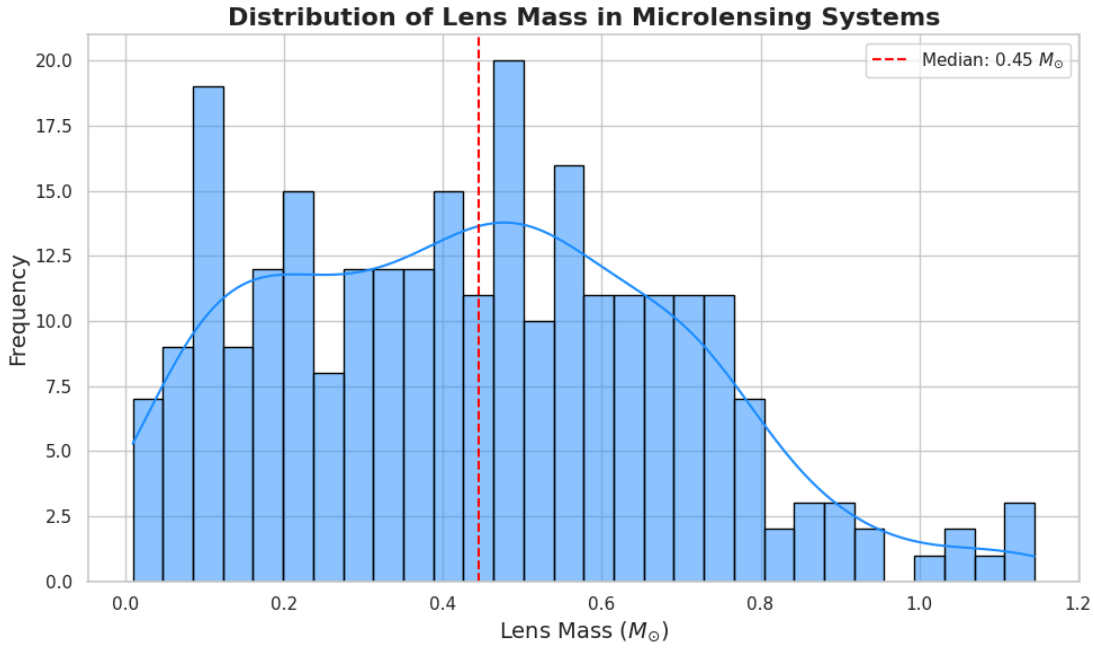


Image 2. Lens' Mass distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [25] in column Lens Mass [Solar mass]

Mass ratio (q)

The mass ratio, beyond its role in the equations, indirectly defines the bending capacity for the planet which will then alter more or less the light curve of the source.
As all the events in [25] are confirmed binary (or multiple) lens events and provide the lens mass and planet mass of the system, q is computed for each event and plotted using Python.

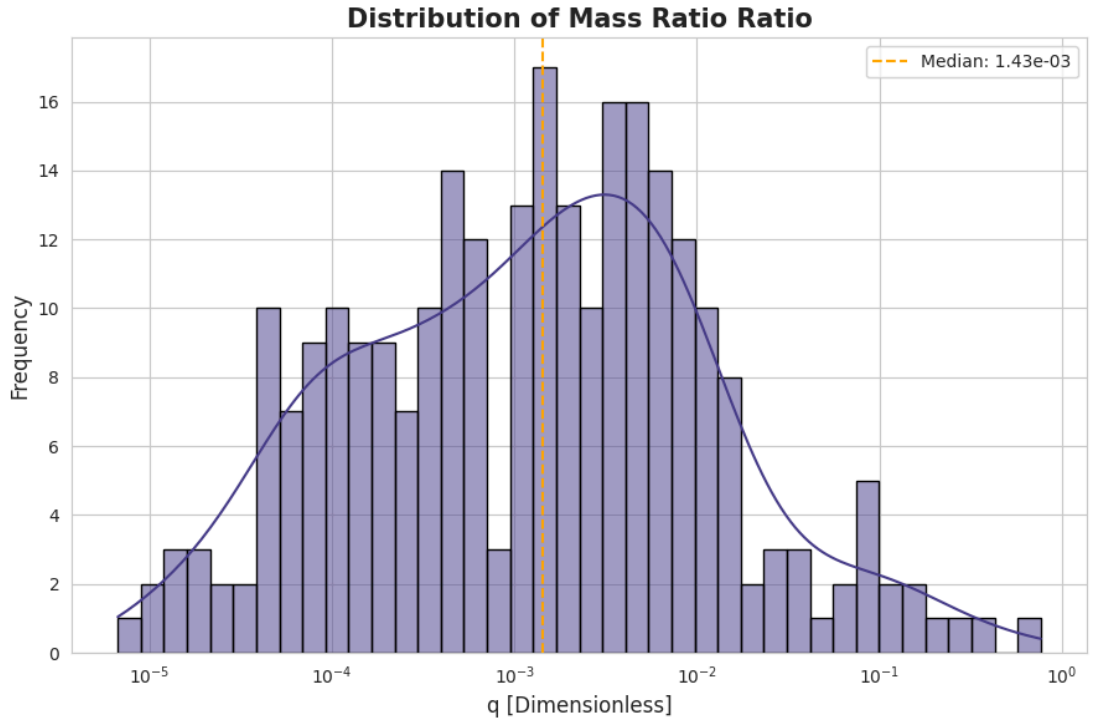


Image 3. Mass Ratio distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [25] in columns Lens Mass [Solar mass] *and* Planet Mass [Jupiter mass]

As seen on the graphic, it can be approximated as a log-normal distribution with a peak at approximately $1.43 \cdot 10^{-3}$ expressed as dimensionless mass ratio.

Distance from lens to source (s)

The distance from lens to source (named D_ls on the equations) is calculated by subtracting the distance to the lens from the distance to the source. In [25] there are only a few observations which specify the distance to source,the resulting distribution may not be fully representative; nevertheless, it constitutes the most reliable observational reference currently available and is used accordingly. I have plotted it with Python resulting in an irregular distribution which tends to 0.5 kpc but it is more or less uniform on every distance as can be seen in below:

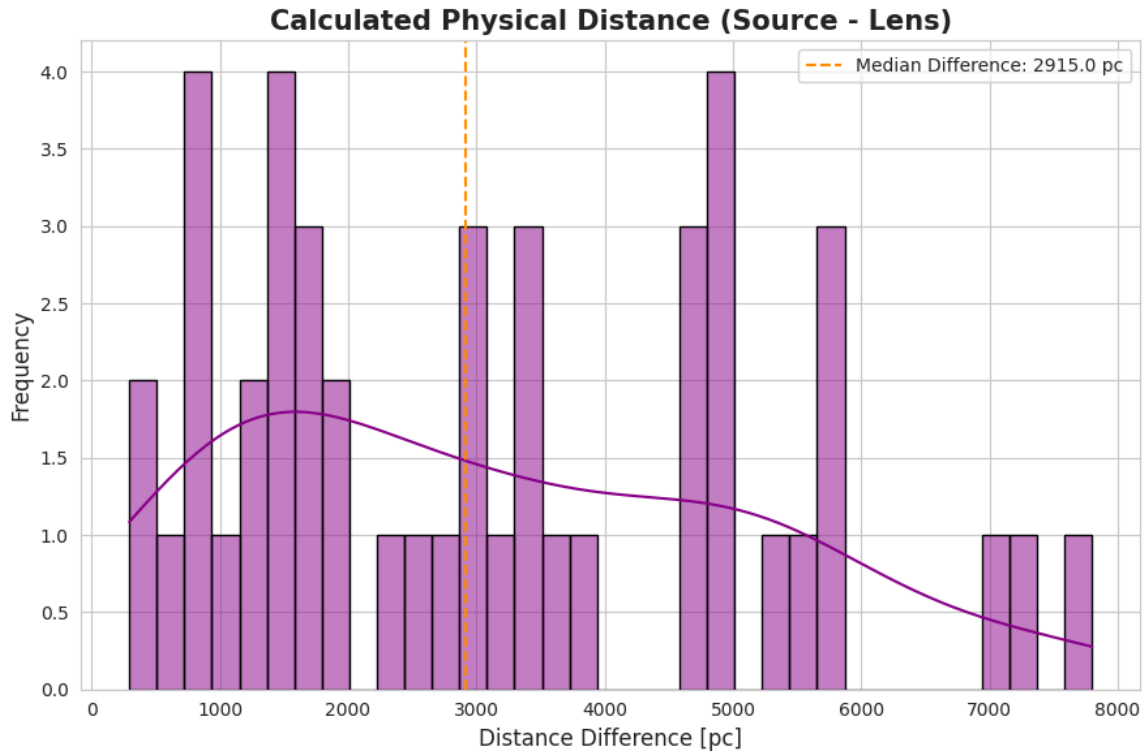


Image 4. Lens to Source Distance distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [25] in columns Lens Distance [pc] *and* Source Distance [pc]

Distance to lens (D_l)

Paczyński suggest in [32] that, as seen in Figure 4 of the paper, the distance from the observer to the lens the curve reflects a prominent peak within the bulge regime at 6 - 7 kpc, in the area where most actual telescopes look, described in the paper as *b* which represents the angle of elevation of the observation by the center of the galactic disk, which is from 3º to 4º as there are many planets (so high microlensing event probability) but not too much galactic dust as there would be at 0º to 2º.

To ensure the validity of these distribution I checked with real data from [25] in the column *Lens Distance [pc]* by plotting it with Python and the results were the following:

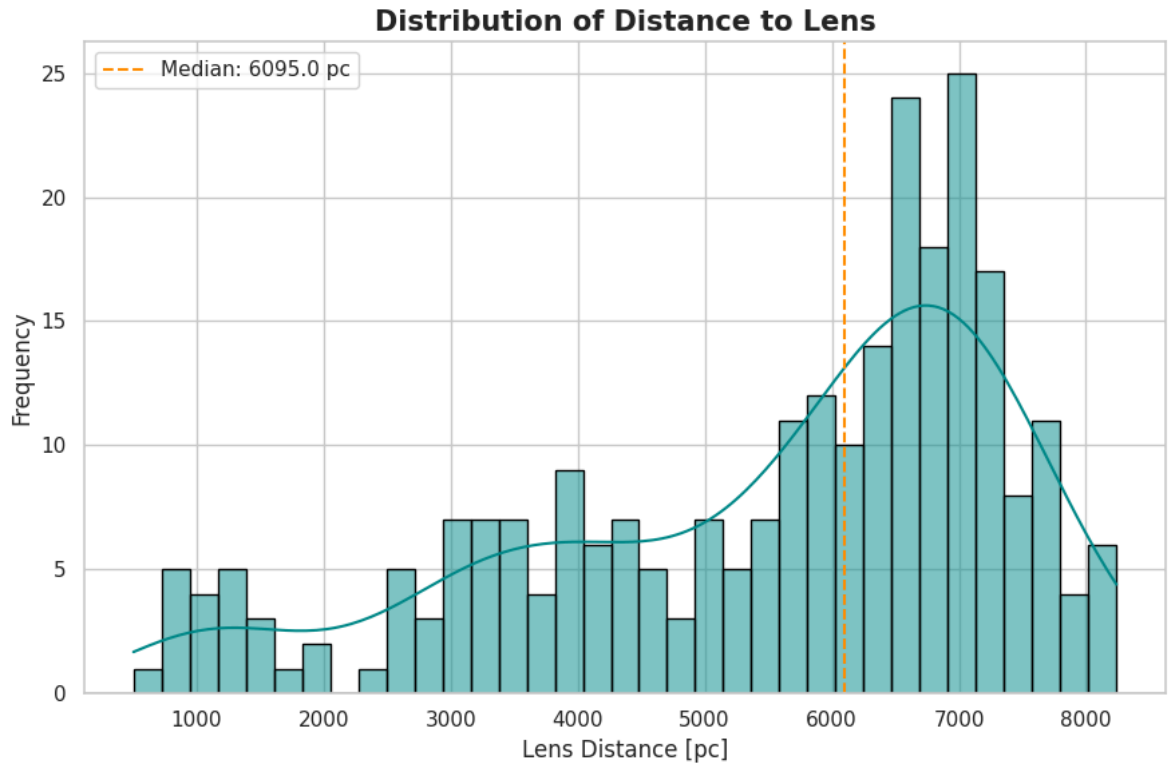


Image 5. Observer to lens distance distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [25] in column Lens Distance [pc]

In the image we can appreciate that from the data at [25] where events from OGLE, MOA, KMTNet and Kepler are published, the distribution of distance to lens is similar to what proposed by Paczyński although with the main peak at 7 kpc.

Also, to see if there is any relation between the lens' mass and the distance from the observer to it I created a coverage plot for these two parameters which demonstrates that there is no apparent relation among mass and distance.

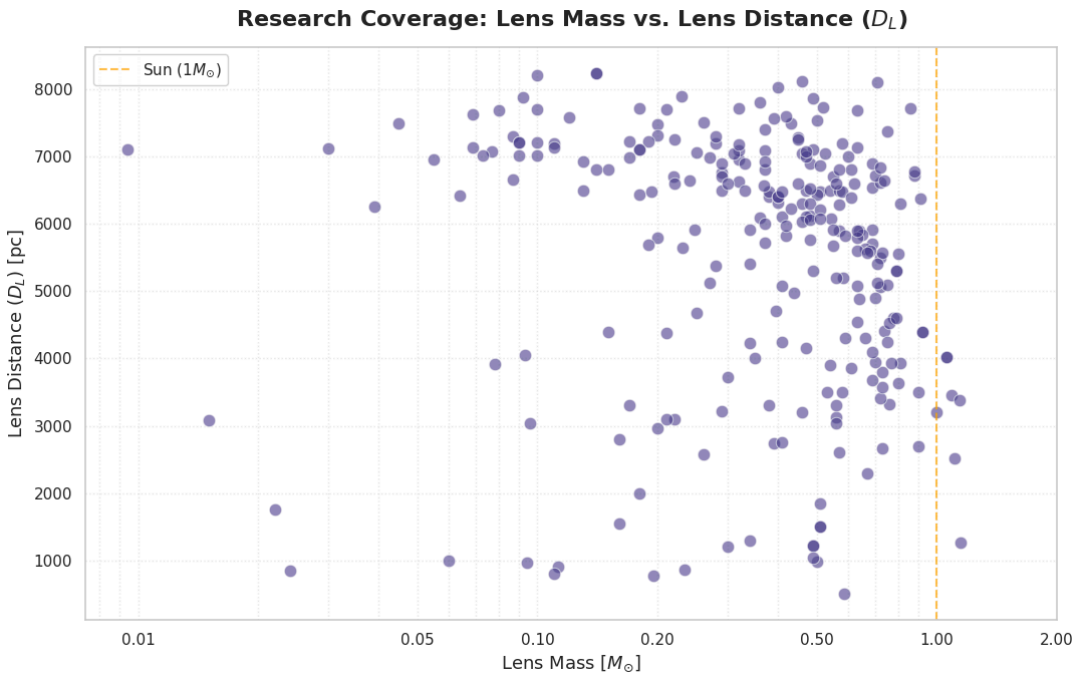


Image 6. Mass and Distance to Lens coverage.Source: Personal elaboration generated with Python from data available at [25] in columns Lens Distance [pc] *and* Lens Mass [Solar mass]

Lens' velocity (v_⊥)

Lens' transversal velocity can't be measured with a simple photometry study which makes it more difficult to find real data. Therefore, a Maxwell-Boltzmann distribution is assumed, with a characteristic speed (mode) of 200 km/s, consistent with [34].

Impact parameter (u0)

Following Paczyński (1986), the impact parameter u0 is theoretically expected to follow a uniform distribution over (0,1), since the probability of a given alignment is proportional to the projected area within the Einstein ring , nevertheless, an examination of the public OGLE-IV survey data [30] resulted with the real distribution of u0 being skewed with a strong concentration near zero and a progressively decreasing tail toward larger impact parameters. This is likely an observational bias: events with u0 values close to zero produce significantly higher peak amplification and are therefore easier to detect with current telescopes.

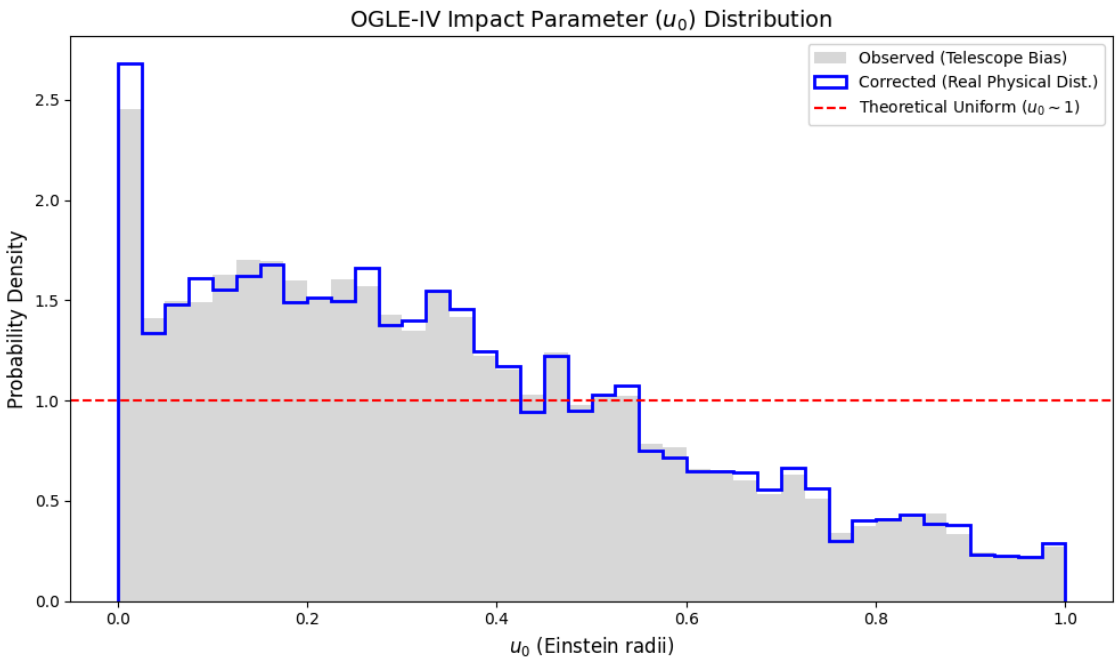


Image 7. Impact parameter distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [30]

Semi-major axis (a)

The semi-major axis is a key definer of the planet's orbit therefore important to define the planet's area of light bending.

The available data in [25] for the semi-major axis and eccentricity, which I will talk about in the next section, is limited so I looked it up on NASA's web and found [26].

In [26] you can find each parameter for every confirmed planet, not only microlensing detected ones. When opening the table you may probably wonder why there are about 40000 rows of planet data with only 6000 planets detected. This is because in that table NASA includes all publications of parameters that have been made for each planet. Filtering by the default parameter set column reduces the table to 6,128 unique planet entries (as of the time of this writing), each corresponding to the best-fitting published model for each confirmed planet.

The problem with this data is that it does not only reflect the microlensing detected parameters, which are different to the ones detected by radial velocity or transit, therefore another filter on only microlensing detected events has to be applied. In transit or radial velocity events the semimajor axis tends to be smaller yet microlensing events usually study higher semi-major axis events, the ones that other methods find it more difficult to understand.

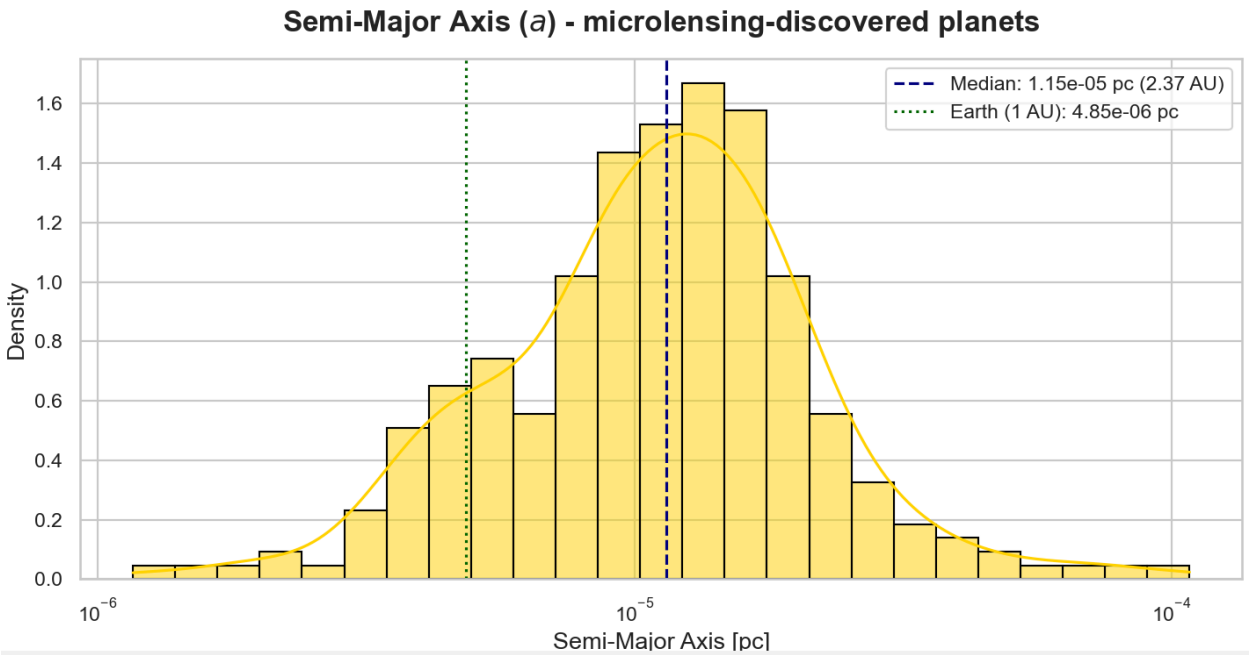


Image 8. Planet's semi-major axis distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [26] in column Orbit Semi-Major Axis [au]

In the plotted data we can see a unimodal distribution, approximately log-normal, peaking at about 10^{-5} pc.

Orbital eccentricity (e)

The orbital eccentricity, as well as the semi-major axis, describes the planet's orbit and, with these two parameters, we could know exactly the apastron and the periastron of each orbit.

As in with the semi-major axis data I am going to use the data found at [26] which will not damage the distribution even though including events not detected via microlensing. [44]

When analysing and plotting the data too many zeros were found, which would mean that the orbit is completely circular, and the cause was investigated. The problem is that many, usually transit, surveys assume a perfect circular orbit to make it easier to calculate any other parameter and, as seen in the plot below, the probably "real" eccentricity is not that far from zero, the assumed error is not that big.

In the plot, these assumed values were excluded by requiring each plotted data to also have presence in the orbital eccentricity error column, ensuring that it has been analyzed and not assumed.

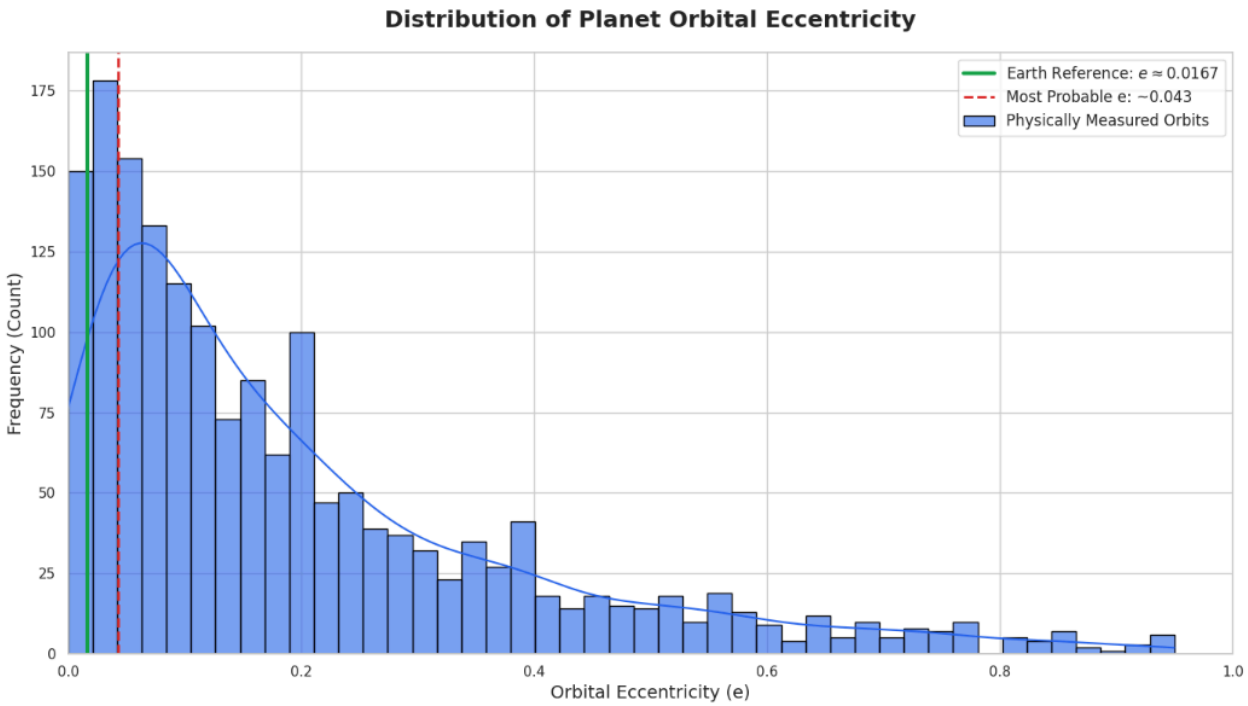


Image 9. Planet's orbital eccentricity distribution in a microlensing event. Source: Personal elaboration generated with Python from data available at [26] in column Eccentricity

We could describe the plot as a long-tailed descending distribution which is probably due to the fact that older planets will always be more affected by their star's gravity and keep turning their orbit more circular and also because it is known that in systems with more than 2 or 3 planets orbits tend to be almost perfect.

Trajectory angle of reference (α_{ref})

The trajectory angle of reference is the most challenging parameter to assign when generating a binary lens dataset.

This angle defines the trajectory of the source through the lens system, which is theoretically random. However, if the source trajectory does not pass through the planet's lensing region, any of our telescopes analysing microlensing would detect the event as a single lens event (although other methods like transit or radial velocity could more easily detect the possibility of a planet in that circumstances).

The effective trajectory angle varies over time as the planet orbits the primary lens. Therefore, from a reference α (which is the one that will be randomized) its trajectory will be computed using the formula so depending mainly on the planet's orbit.

Source baseline magnitude (I_s)

The source baseline magnitude represents the apparent brightness of the background source star in the absence of any lensing effect. Since microlensing surveys preferentially monitor dense stellar fields toward the Galactic Bulge, the source population is dominated by bulge red clump giants and subgiants, which concentrate around I_s ≈ 19.5 – 20 mag. An examination of the public OGLE-IV survey data [30] reveals a distribution peaking near this range, with a declining tail toward fainter magnitudes (I_s > 21) reflecting both the intrinsic stellar luminosity function and the survey's detection efficiency drop at low signal-to-noise, and a sharp cutoff at bright magnitudes (I_s < 14) imposed by detector saturation.

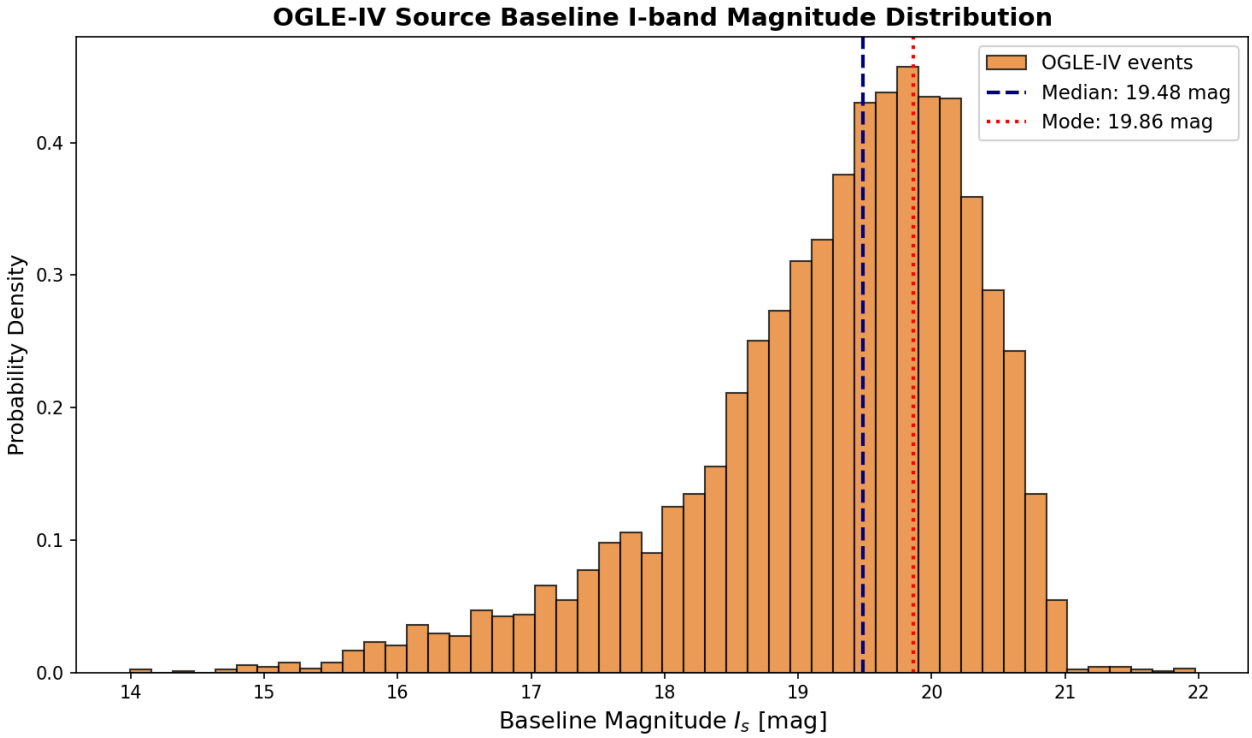


Image 10. OGLE-IV survey extracted source baseline I-band magnitude distribution. Source: Personal elaboration generated with Python from data available at [30]

Unlike every other parameter described in this section, the source baseline magnitude does not describe the lens–source geometry and does not enter the amplification equations at all: changing I_s does not change the shape of the event in the slightest. Its role is a different one. The equations derived earlier produce the amplification A(t), which is a dimensionless number (it only says how many times brighter the source appears than usual). A telescope, however,

does not record "times brighter"; it records brightness on the magnitude scale. The source baseline magnitude is precisely what allows one to be converted into the other.

The magnitude scale is inherited from antiquity, when stars were ranked from first magnitude (the brightest) down to sixth (the faintest visible), which is why the scale runs backwards: a larger magnitude means a fainter star. Pogson made this precise by defining that a difference of exactly 5 magnitudes corresponds to a factor of exactly 100 in flux [33]. Each single step in magnitude is therefore a factor of 100^(1/5) ≈ 2.512 in received light. Written as a formula, two stars of flux F_1 and F_2 differ in magnitude by

$$m_2 - m_1 = -2.5 \cdot \log(F_2 / F_1)$$

The constant 2.5 is exactly what enforces Pogson's definition: if one star is 100 times brighter, then log(100) = 2 and the magnitude difference is −2.5 × 2 = −5, which is the required 5 magnitudes. The minus sign encodes the inverted scale: the brighter star ends up with the lower magnitude number.

Applying this to a microlensing event is immediate, because the amplification is itself a ratio of fluxes: A(t) is by definition the flux at time t divided by the source's unlensed baseline flux. Comparing the event with its own baseline (that is, taking F_2 = F(t) and F_1 = F_source in the relation above) the flux ratio is simply A(t), and the light curve in magnitudes becomes

$$I(t) = I_s - 2.5 \cdot \log A(t)$$

The interpretation is straightforward: the amplification A(t) sets the shape of the curve, while I_s sets the level at which that shape sits on the brightness axis. When the source is magnified (A > 1) the logarithm is positive, the magnitude decreases, and on the inverted scale the star correctly appears brighter.

This is why a value of I_s must be drawn for every generated event whenever the dataset is produced in magnitudes rather than in amplification: each real event happens to a different source star, of different intrinsic brightness, so each synthetic event is assigned its own I_s from the distribution above. The choice matters beyond mere realism. As will be seen in the following section, the photometric uncertainty of a measurement depends directly on how much light the detector actually receives, so the brightness assigned to an event is also what decides how noisy its light curve will be. In the idealised case considered here, where every photon is assumed to come from the source, that brightness is simply I_s. In a real observation, however, the aperture also collects light from other stars, and it is then the total recorded brightness, not the source's alone, that governs the noise. This is one of the reasons why blending, introduced later, cannot be ignored.

This idealised conversion, in which all the recorded light is assumed to come from the source, is the one used to generate clean magnitude light curves. It is not, however, what a real telescope records in the crowded fields towards the Galactic Bulge, where the aperture inevitably collects light from other stars as well. When realistic OGLE-IV data is reproduced, the relation is therefore generalised to account for this blending, as described later.

Observational imperfections

All the light curves that could have possibly been generated so far are mathematically perfect: the magnitude is known at every instant and follows the exact theoretical curve. Real observations are never like this. A real telescope introduces three separate imperfections that a realistic synthetic dataset must reproduce:

- Photometric noise: every measurement of the source's brightness carries an uncertainty. The measured magnitude is scattered around its true value, and this scatter grows for fainter stars, which receive fewer photons.
- Observing cadence: a telescope does not measure the source continuously. Observations are taken at irregular moments and, more importantly, there are long periods when the target simply cannot be observed at all.
- Blending: the measured brightness is not that of the source alone. In dense stellar fields the photometric aperture also collects a constant amount of light from the lens and from unresolved neighbouring stars, which makes the baseline brighter than the source and dilutes the amplification peaks.

If a machine learning model is trained only on perfect curves, it will perform poorly on real data, which is noisy, full of gaps, and diluted by the light of neighbouring stars. To avoid this, instead of inventing arbitrary imperfections, all three were extracted empirically from real surveys and then reproduced in the synthetic data.

OGLE-IV

Data source

OGLE (Optical Gravitational Lensing Experiment) is a long-running survey, operated by the Warsaw University Astronomical Observatory, which monitors the Galactic Bulge in search of microlensing events. Its fourth phase, OGLE-IV, regularly observes over a billion stars and is one of the largest sky variability surveys in the world [43]. All the imperfections reproduced in this part of the project are extracted from it.

The survey makes its results public through two different products, and the distinction between them matters throughout this section. The first is the raw photometry: the actual sequence of measurements taken for each event. The second is the event catalogue: not measurements at

all, but the fitted parameters that best reproduce each event's light curve. One is what the telescope recorded; the other is what was inferred from it.

The photometry is released through the Early Warning System (EWS), which announces every microlensing event as it is detected and publishes its complete light curve [29]. A list of 17172 OGLE-IV event identifiers (years 2011–2025) was collected, and a random sample of 3 000 of them was downloaded. For each event, the survey provides a file listing, for every single observation: the time (Heliocentric Julian Date), the measured I-band magnitude, and its reported uncertainty σ_I. Pooling all 3 000 events gives roughly 7.6 million real observations. This is the raw material for the noise and cadence models described below, and also for the observed baseline magnitudes used later to verify the blending model.

The event catalogue [30] serves a different purpose. It contains one row per analyzed event, listing the parameters (among them the source magnitude I_s and the blend fraction f_s) obtained by fitting a model light curve to that event's photometry. Of these, 5760 events provide a usable pair of both values, and they are the raw material for the blending model. Because these numbers are fitted rather than measured, they carry the uncertainties of that fit, which is why some of them can fall slightly outside their physically allowed range.

Photometric noise

Because every observation already comes with its own reported uncertainty σ_I, the noise can be characterised directly. Plotting σ_I against the magnitude I of every observation reveals a clear trend: the uncertainty is small and roughly constant for bright stars, and rises steeply for faint ones.

This behaviour corresponds to the two physical sources of noise combined in quadrature:

- A photon-noise term that dominates at the faint end and grows exponentially with magnitude, and
- A constant systematic floor noise that dominates at the bright end and is set by the instrument and reduction pipeline.

The photon noise exists because light does not arrive as a smooth flow but as individual photons, one by one, at random instants. Measuring a star's brightness is therefore like counting raindrops in a bucket to estimate how hard it is raining: if only a few drops fall, the estimate is rough, but if thousands fall, the random arrival of each individual drop barely matters and the estimate becomes precise. Statistically, if a detector collects N photons, the random fluctuation of that count is $\sqrt{N}$ [13], so the error relative to the total signal is $\sqrt{N}/N = 1/\sqrt{N}$. This means that the more photons are collected, the smaller the relative error becomes. Now, the magnitude

scale is defined so that brightness, and therefore the number of photons collected, decreases exponentially with magnitude, following $N \propto 10^{(-0.4 \cdot I)}$ [33]. Combining both facts, the error in magnitude follows $\sigma \propto 1/\sqrt{N} \propto 10^{(0.2 \cdot I)}$, and squaring it gives $\sigma^2 \propto 10^{(0.4 \cdot I)}$. This is why the photon-noise term grows exponentially with magnitude, and it is the exact shape used in the model below: nothing in it is arbitrary, it follows directly from photon counting and the definition of the magnitude scale.

The floor noise, in contrast, has nothing to do with the star's brightness. It comes from the detector and the data-reduction pipeline (effects such as background subtraction or calibration limits) which introduce the same small error regardless of how bright the observed star is. It therefore acts as a constant that sets the best precision the survey can ever reach, and it becomes the dominant contribution only for the brightest stars, where photon noise is negligible.

Since the two sources are independent, they are combined by adding their variances (their squares) and taking the square root, which is the standard way of combining independent uncertainties.

This yields the complete noise model: $\sigma(I) = \sqrt{\sigma_{floor}^2 + \sigma_{phot0}^2 \cdot 10^{(0.4 \cdot (I - 18))}}$

where σ_{floor} is the constant instrument floor and σ_{phot0} is the photon noise at the reference magnitude $I = 18$ (at that magnitude the exponential factor equals 1, so the whole term reduces to σ_{phot0}).

To measure these two parameters from the data, the 7.6 million observations were sorted by magnitude and grouped into narrow bins 0.2 mag wide. For each bin the median value of σ_I was taken, which condenses the enormous scatter of individual points into a single clean curve of "typical error versus magnitude". The model was then fitted to that curve, adjusting σ_{floor} and σ_{phot0} until it matched the observed points as closely as possible. The fit was performed in logarithmic space so that every magnitude bin contributes equally, regardless of how many observations it contains. The result is $\sigma_{phot0} \approx 0.026$ mag, which is well determined because faint stars, where photon noise dominates, are abundant in the sample. The floor noise, on the other hand, can only be given as an upper limit: bright stars are too rare in the data to constrain it precisely, so it has been pushed to the smallest value it was allowed to take ($\sigma_{floor} \approx 0.0001$). This has no practical consequence, since photon noise dominates across the whole magnitude range of interest.

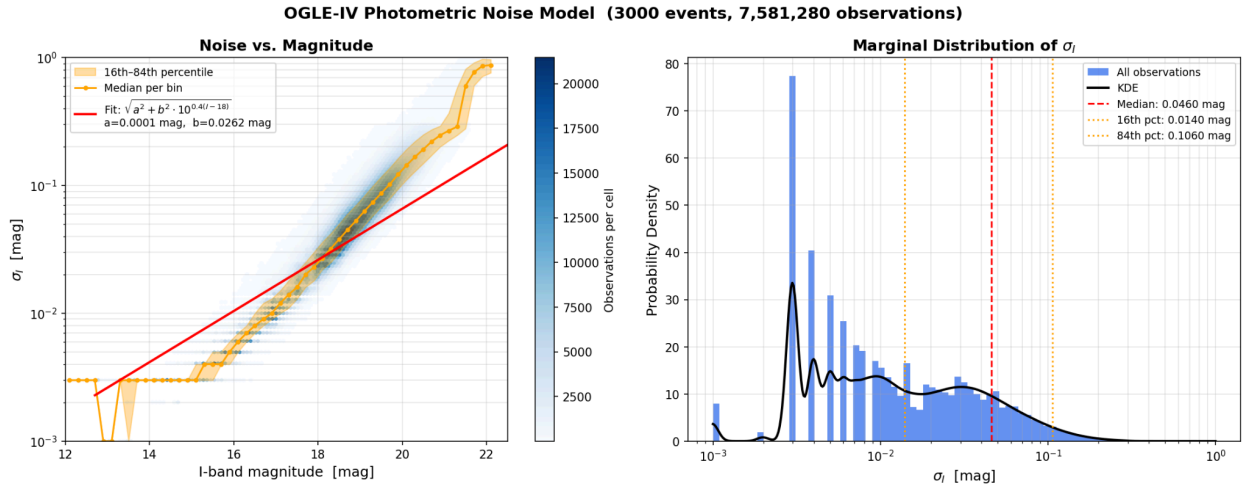


Image 11. Photometric noise model from OGLE-IV survey. Source: Personal elaboration generated with Python from data available at [29]

Once the model is known, applying it to any synthetic dataset is straightforward. For each point of a generated light curve, of magnitude I, a random value is drawn from a Gaussian distribution centered on zero and with standard deviation σ(I), and this value is added to the true magnitude. Bright points are therefore barely displaced, while faint points are scattered much more, exactly reproducing the behaviour of real observations. Because the whole procedure depends only on the two fitted parameters, adapting it to a different survey in the future—for instance, the Roman Space Telescope [24]—would only require repeating the extraction on that survey's photometry to obtain its own σ_floor and σ_phot0, leaving the rest of the method unchanged.

Observing cadence

The second imperfection concerns not the value of each measurement but its timing. The noise model described *how wrong* each brightness measurement is; the cadence describes *when* the measurements happen and, above all, which moments are not observed at all. A real light curve is therefore not a smooth, continuous line but a set of points scattered irregularly in time, with large stretches simply missing. Reproducing this is just as important as reproducing the noise: a model trained only on evenly and completely sampled curves would fail on real data, which is full of gaps.

The gaps exist for purely practical reasons. A ground-based telescope cannot observe a target during the day, when the Sun is up; it cannot observe when the sky is cloudy; and it cannot observe the Galactic Bulge at all during the part of the year when that region of the sky is

behind the Sun. These three limitations act on very different time scales between hours, nights, and months, and together they shape the entire observing rhythm of the survey.

To characterise this rhythm, the time gap Δt between each pair of consecutive observations of the same event was computed, for all 3 000 events, giving about 7.58 million gaps in total. Sorting these gaps by duration reveals two completely separate regimes, split at roughly 100 days: the normal within-season sampling, and the long seasonal interruptions.

The within-season gaps (Δt < 100 days) make up 99.9% of all gaps and represent the everyday observing pattern, with a median spacing of about 57 minutes. Within this regime there are two clearly distinct behaviours. About 76% of the gaps are shorter than half a day: these are several observations taken during the same night, with a typical spacing of roughly 33 minutes. The remaining 24% are longer than half a day and cluster around one day, reflecting the natural night-to-night rhythm of observing the target once each clear night. The seasonal gaps (Δt ≥ 100 days), on the other hand, account for only 0.1% of the gaps but are enormous, with a mean length of about 194 days. They correspond to the months in which the bulge is unobservable, and from them it follows that the region is visible only about 47% of the year, roughly 171 days out of 365.

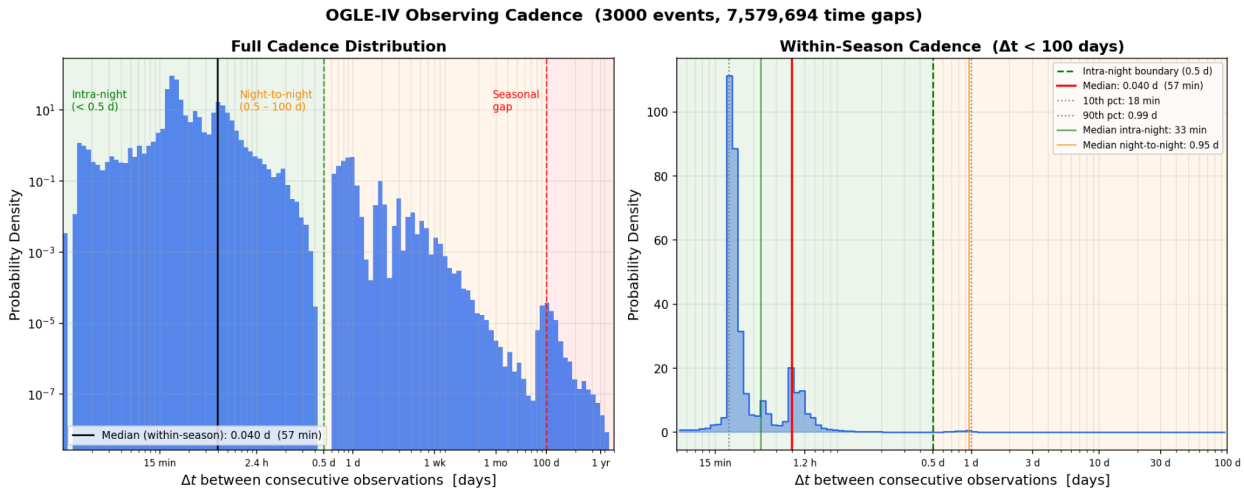


Image 12. Observing cadence model from OGLE-IV survey. Source: Personal elaboration generated with Python from data available at [29]

Applying this pattern to a synthetic light curve means deciding which of its points would actually have been observed. A realistic sequence of observation times is built by repeatedly drawing gaps at random from the measured within-season distribution and adding them up, producing a schedule with the same irregular, clustered rhythm as a real OGLE-IV event. Each point of the generated curve is then kept only if one of these observation times falls close enough to it;

every other point is marked as missing. The result is a curve sampled at the same uneven pace as a genuine observation, with realistic short gaps between points. It is worth being precise about one limitation here: this schedule is drawn only from the within-season distribution and spans the duration of a single event (a few Einstein times), so it faithfully reproduces the hour-to-day sampling and its gaps, but it does not stamp the multi-month seasonal blackouts onto individual curves. This is a deliberate and reasonable choice, since a single microlensing event is normally much shorter than one observing season; the seasonal gaps are therefore characterised as a real property of the survey rather than applied to every light curve. As with the noise, the whole procedure depends only on the measured gap distribution, so adapting it to a different survey in the future such as the Roman Space Telescope, which observes from space and would have a very different cadence [24], would simply require repeating the extraction on that survey's data.

Blending

The conversion $I(t) = I_s - 2.5 \cdot \log A(t)$ assumes the source is the only star contributing light to the measurement yet in the dense fields toward the Galactic Bulge this is almost never true. What a telescope records is not a single isolated star but a small blurred patch of sky, and within that patch there is usually more than one object: the lens itself, if it shines, and any unresolved neighbouring stars that happen to fall within the same seeing disk [21]. The light of all of them is added together into a single measurement. Crucially, only the source is magnified during the event therefore everything else contributes a constant, unmagnified amount of light. This extra, steady contribution is called the blend.

The observed flux is therefore the magnified source plus a constant term [34]:

$$F_{obs}(t) = A(t) \cdot F_{source} + F_{blend}$$

To describe how much of the light actually belongs to the source, one defines the source flux fraction, the fraction of the baseline light contributed by the lensed star [21]:

$$f_s = F_{source} / F_{baseline} = F_{source} / (F_{source} + F_{blend})$$

where $F_{baseline} = F_{source} + F_{blend}$ is the total flux recorded when no lensing is happening ($A = 1$). By construction, f_s lies between 0 and 1: a value of 1 means no blending at all, while a value close to 0 means the source is only a tiny fraction of the light in the aperture.

Dividing the observed flux by the baseline flux, the absolute fluxes cancel and what remains is an observed amplification that is simply a weighted average of the source's amplification $A(t)$ and the blend's, which is always 1 because the blend never brightens:

$$A_{obs}(t) = f_s \cdot A(t) + (1 - f_s)$$

Converting this to magnitudes gives the light curve that a real telescope would record:

$$I(t) = I_{base} - 2.5 \cdot \log(f_s \cdot A(t) + (1 - f_s))$$

where I_{base} is the observed baseline magnitude, the brightness actually recorded at $A = 1$. Setting $f_s = 1$ (no blend) recovers the idealised relation $I(t) = I_s - 2.5 \cdot \log A(t)$, as it must.

Blending has two consequences. First, the baseline is brighter than the source alone, simply because extra light has been added to the aperture. Second, and more importantly, the peaks are diluted: since A_{obs} is an average of $A(t)$ and 1, it always lies between them, so the observed amplification is always weaker than the true one. The smaller f_s is, the more the event is washed out, and in the limit $f_s \to 0$ the event disappears entirely. This matters far beyond the baseline level, because the dilution is not a simple rescaling: it changes the shape of the curve, compressing the sharp features that betray a binary lens relative to the overall bump.

Both parameters come from the same product of the survey: the OGLE-IV event catalogue [30], which reports, for every analyzed event, its fitted source magnitude I_s and its blend fraction f_s. For each synthetic event, one real catalogue entry is chosen at random and its two values are taken together, as a pair. Of the catalogue, 5 760 events provide a usable pair, with a median $f_s \approx 0.70$. The observed baseline is then not drawn at all, but follows from them: since $f_s = F_{source} / F_{baseline}$, applying Pogson's relation [33] to that flux ratio gives

$$I_{base} = I_s + 2.5 \cdot \log f_s$$

and because $f_s \leq 1$ the logarithm is negative, so I_{base} is smaller, brighter, than I_s, exactly as the added blend light requires.

Drawing the two values as a pair is essential, and not merely convenient. In the real catalogue the source magnitude and the blend fraction are not independent: brighter sources are measurably less blended, since a bright star dominates the light in its own aperture, while a faint one is easily outshone by its neighbours. The effect is clear in the data:

Source magnitude I_s	median f_s
12 – 17 (bright)	0.880
17 – 18	0.796
18 – 19	0.721
19 – 20	0.684
20 – 21 (faint)	0.657

Sampling I_s and f_s from their two distributions independently would reproduce each one correctly but destroy the relationship between them, generating impossible combinations such as very bright yet heavily blended sources. Resampling whole catalogue rows keeps every synthetic event internally consistent: its source magnitude, its blend fraction and its baseline all belong to the same real event.

One caveat must be noted. Physically f_s cannot exceed 1, since that would require a negative blend flux. Yet about 8% of the catalogue entries have a fitted f_s slightly above 1. These values are not real: f_s is not measured directly but fitted to noisy photometry, and for events with very little blending the noise can push the best-fit value just past the physical boundary—much as a noisy scale can read a small negative mass for a nearly weightless object. They are nevertheless kept rather than discarded or clipped, because removing them would preferentially eliminate the least-blended events and bias the distribution toward more blending than actually occurs.

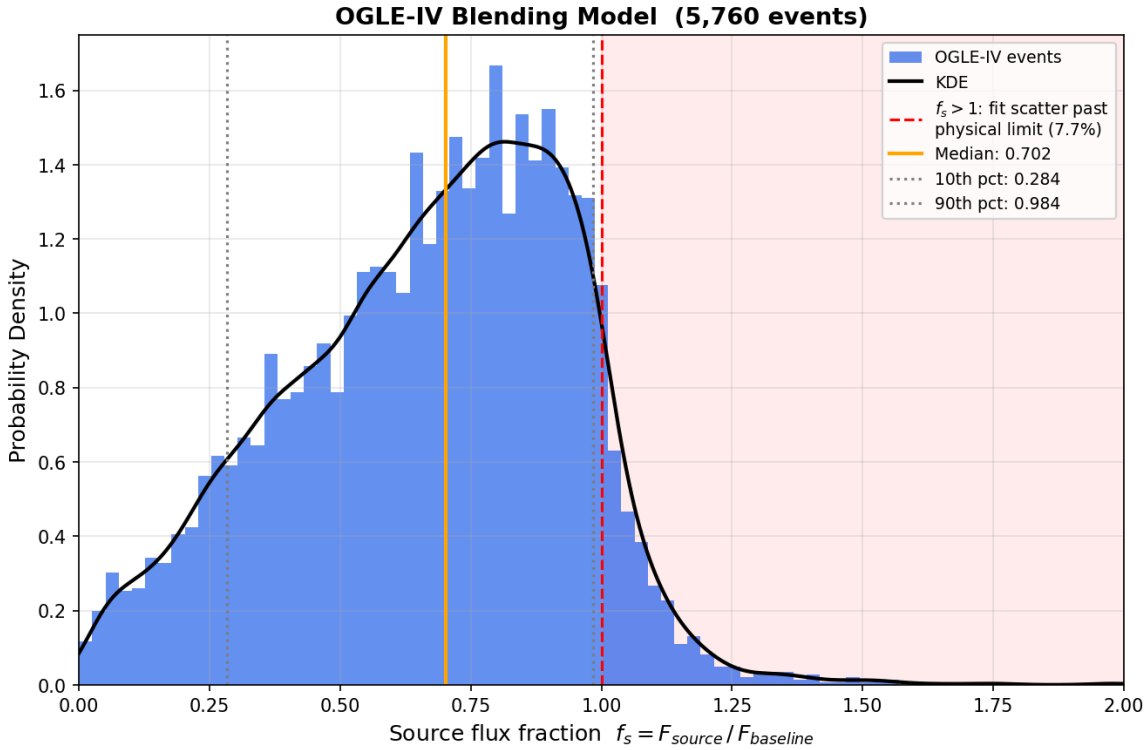


Image 13. Distribution of the OGLE-IV blend fraction f_s. Source: Personal elaboration generated with Python from data available at [30]

Finally, the resulting model can be tested against data that was never used to build it. The values of I_s and f_s come from the fitted catalogue, so the baseline they imply is a prediction. That prediction can be compared with the baseline measured directly from the raw photometry,

an entirely separate product of the survey. To measure it properly, one baseline is taken per event, as the median magnitude of that event's own light curve; pooling all the individual observations instead would be biased twice over, since heavily-monitored events would count thousands of times while sparsely-observed ones would barely count at all, and the magnified points themselves would drag the distribution towards brighter values. A median per event removes both effects, being insensitive to the small fraction of points lying above the baseline.

Measured this way, the observed baselines of real OGLE-IV events peak near 18.94 mag, while the catalogue independently implies 18.86 mag which results to an agreement of 0.08 mag, holding not only at the median but across the whole distribution. Since the two figures come from unrelated data products, this constitutes a genuine, independent confirmation that the blending model reproduces the brightnesses the survey actually records.

Machine Learning

To develop the final product which will predict whether a microlensing event contains a planet, more than one machine learning algorithm has to be developed. In this section, the theoretical foundations of machine learning are introduced, followed by a description of the methodology used in this work.

What machine learning is

Machine Learning is a subfield of artificial intelligence whose goal is to develop algorithms that improve their performance on a given task through experience, without being explicitly programmed for each case [22]. A standard formalization defines it as: a computer program is said to learn from experience E with respect to a task T and performance measure P, if its performance at T, as measured by P, improves with experience E [37].

Unlike traditional programming, where an expert hand-crafts a list of rules, machine learning lets large amounts of data determine the solution. This is especially valuable for tasks that are intuitive for humans but formally impossible to describe rule by rule, such as recognizing patterns in complex signals [20].

Learning tasks are generally categorized into three branches:

- Supervised learning: The model is trained on labeled examples (input-output pairs) to learn a mapping that generalizes to unseen data.
- Unsupervised learning: The model extracts hidden structure from unlabeled data, through clustering or dimensionality reduction.
- Reinforcement learning: An agent learns an optimal sequence of decisions by interacting with an environment and receiving reward or punishment signals [37].

This work exclusively uses supervised learning, as all synthetic training samples carry a known label.

The three components of machine learning

Every machine learning algorithm, regardless of its complexity, is built from three fundamental components: representation, evaluation, and optimization [37]. Understanding what each one does is key to understanding how a model actually learns.

- Representation defines the space of possible solutions the model can explore — in other words, what kind of answer it is even capable of giving. Choosing the wrong representation means the correct solution is simply unreachable, no matter how much data or computing power is available.
- Evaluation is how the model measures its own error. After making a prediction, the model needs a way to know how wrong it was and by how much. This is done through a metric.
- Optimization is the mechanism by which the model improves. Once the evaluation tells the model how wrong it is, optimization determines what to change and in which direction to reduce that error.

Together, these three components define not just how a model is built, but why it works or why it fails.

The generalization process

The ultimate goal of machine learning is generalization: the ability to perform correctly on new, unseen, data rather than only on training data [5].
Since a model cannot be evaluated on data it has seen during training, the available dataset is split into a training set and a held-out test set, typically keeping between 10% and 20% of samples for evaluation.

Successful generalization requires overcoming two main challenges: underfitting and overfitting
If the model is too simple for the data it processes, the output will never be sufficiently accurate. Otherwise, if the model is too complex, it will memorize noise and spurious patterns specific to the training samples, achieving high accuracy on similar data but failing to generalize to unseen examples. [36]
Managing this balance relies on the bias-variance tradeoff, considered a universal principle in the field. Bias represents error arising from overly simplistic assumptions; high bias leads to

underfitting. Variance represents error arising from excessive sensitivity to fluctuations in the training set; high variance leads to overfitting. Optimal generalization is achieved at a moderate level of complexity where the sum of bias and variance is minimized.

Finally, the No Free Lunch Theorem establishes that no single algorithm is universally superior across all possible tasks, meaning every learner must embody specific prior knowledge or an inductive bias to generalize beyond its given data [5].

Types of supervised learning models

Within supervised learning—where a model learns from examples that are already labelled with the correct answer—there is no single kind of model, but rather several families that differ in how they draw the boundary between classes and in the assumptions they make about the data. The most common ones are the following:

Linear and logistic models, the simplest ones, which separate the classes with a straight boundary and, in the logistic case, return the probability of belonging to each class. They are fast and easy to interpret, but they can only succeed when the classes are already almost separable by a straight line, and they fail when the distinguishing pattern is more complex.

k-Nearest Neighbours (k-NN) takes the opposite approach and does not really learn a function at all. It simply stores every training example, and to classify a new one it looks at the few stored examples closest to it and takes the majority label among them [4]. It is very intuitive, but it becomes slow when there are many examples and loses reliability when each example is described by a large number of values.

Support-Vector Machines (SVM) look for the boundary that leaves the widest possible margin between the two classes, which tends to generalise well; with the so-called kernel trick, that boundary can also be curved instead of straight [3]. They perform strongly when data is limited and the classes are reasonably well separated.

Decision trees and random forests classify by asking a chain of simple yes/no questions about the data ("is this value above that threshold?"). A single tree is easy to read but tends to overfit, so in practice many trees are trained on random variations of the data and their votes are averaged, forming a random forest that is far more robust [1]. These models work naturally on data organised as a table of independent features.

Neural networks are built from many simple units arranged in successive layers, which together can approximate extremely complex relationships and, crucially, can learn their own

useful features directly from the raw data instead of relying on features designed by hand [18]. This family has several variants suited to different kinds of data. The basic form, the fully-connected network, links every input to every unit and makes no assumption about the structure of the data. The convolutional network (CNN) is specialised for data in which meaningful patterns are local and can appear at different positions, such as images or time-ordered sequences. The recurrent network, and in particular its Long Short-Term Memory (LSTM) version, processes a sequence step by step while keeping an internal memory, which makes it well suited to long sequences with strong temporal dependencies [12] [9].

Each of these families carries a different built-in assumption about what the data looks like, and choosing a model means matching that assumption to the structure of the problem at hand. The following section applies exactly this reasoning to the classification of microlensing light curves.

Model

Task

The model built in this work addresses the most fundamental question that can be asked about a microlensing light curve: was it produced by a single lens system, or by a binary one? Formally this is a binary classification problem (given the shape of a light curve, the model must decide between two categories, single-lens or binary-lens). This question is the very heart of the project rather than a side result, because the distinction between the two cases is exactly what separates an ordinary lensing event from one that may hide a companion such as a planet: a second mass betrays its presence only through the subtle distortions it adds to the curve. It is also the point at which the entire synthetic-data pipeline developed in the previous sections (the physical light-curve models, the parameter distributions, and the OGLE-IV imperfections) is finally put to use, since it is precisely that data that trains and tests the model.

The model is developed in two versions, trained and evaluated on two different kinds of data, and the contrast between them is deliberate. The first version, referred to as Simple, works on perfect light curves (curves generated without photometric noise and without cadence gaps). Its purpose is not to be realistic but to measure how well the two classes can be told apart under ideal conditions, where every point of the curve is known exactly. It therefore establishes an upper bound: the best performance any model of this kind could hope to reach if the observations were flawless. The second version, referred to as Real, faces the situation an actual survey would present (light curves carrying the OGLE-IV photometric noise, cadence gaps and blending described earlier).

Comparing the two versions is what gives the exercise its meaning. The Simple model shows how separable single and binary events are in principle; the Real model shows how much of that separability survives once the data is degraded to the level of genuine observations. The gap between them is not a shortcoming of the method but a direct, quantitative measurement of the cost of realism; of how much information the noise, the gaps and the blending actually destroy. As will be shown, that cost is severe, and understanding why becomes the central scientific result of this section.

Type

The type of model chosen for this task is a Convolutional Neural Network (CNN), a variant of neural network specifically designed for data in which the meaningful patterns are local and can appear at different positions. A light curve is exactly this kind of data: a one-dimensional sequence of brightness values ordered in time, in which the signature of a lens is a particular shape that may sit anywhere along the curve. CNNs were introduced precisely to handle images, speech and time series of this nature [17].

The core idea of a CNN is the filter, which can be pictured as a small transparent stamp with a shape drawn on it—for instance, a sharp spike. The network slides this stamp step by step along the whole light curve and, at each position, checks how well the curve matches the shape on the stamp, "lighting up" wherever it finds a match. Each filter is therefore a small pattern-detector, and a single convolutional layer applies many of them at once, each hunting for a different feature [19]. Because the very same stamp is used at every position, a feature is recognised no matter where it occurs (the network learns what a spike looks like only once and then finds it anywhere along the curve). This property, known as translation invariance, is what makes a CNN so well suited to signals whose important events are not always in the same place.

These filters are then stacked in successive layers, and this is where the real power appears. In much the same way that reading is learned by first recognising strokes, then letters, then whole words, the first layers of a CNN detect very small features such as slopes and edges, while deeper layers combine them into larger and more meaningful shapes, until the network can recognise a complete morphology such as a caustic crossing. Between these layers, a pooling operation repeatedly summarises the signal by keeping the information that a strong feature was found somewhere in a region while discarding its exact location, which both compresses the data and reinforces the position-independence of the detection.

This is what makes a CNN the natural choice for distinguishing single-lens from binary-lens curves. A single-lens event has one smooth, symmetric brightness peak, whereas a binary event adds sharp, localised features (extra spikes, asymmetries or secondary peaks) which are exactly the kind of local patterns a CNN is built to detect, wherever they happen to fall along the curve. The alternative models are less suited to the raw curve: the classical methods described earlier treat the four hundred points of a curve as an unordered list of independent values, so they ignore the very order that gives the curve its shape, and they can only work once a person has decided in advance which numerical features to measure. A fully-connected network keeps

the order but must learn every feature separately for each possible position, and a recurrent network, although designed for sequences, is heavier and better justified for very long sequences with long-range dependencies than for the short, fixed-length curves used here. The CNN, by contrast, learns the relevant shapes on its own and locates them automatically, which is also why convolutional networks have become a standard and highly effective tool for time-series classification in general [15].

It is worth noting that machine learning has already been applied to the classification of microlensing events, for example by combining manually engineered features with a random-forest classifier [8]. The approach taken here differs in emphasis rather than in kind. Both a convolutional network and a classical model must be given the light curve in some prepared form as neither reads it untouched. The real distinction lies in how much of the analysis the person performs in advance. A classical model is handed the curve already reduced to a fixed list of hand-chosen summary numbers, and can never look beyond them: a human has decided what to measure and discarded the rest. A CNN, by contrast, is given the curve as a full-length sequence (together, as the next section will show, with a few physically-motivated helper channels) and is left to discover for itself which local shapes within those sequences matter, where they occur, and how to combine them. It is this ability to learn its own detectors from the full curve, rather than any absence of data preparation, that motivates its choice.

That difference, however, raises a fair question: if the right features were designed by hand, could a simpler classical model match the CNN? To answer it honestly, this work also builds that classical alternative as a check. A single physically-motivated feature (how badly a pure single-lens model fails to fit each curve, described in detail in a later section) is computed for every light curve and fed to a gradient-boosted tree (GBT). As said in previous sections, a tree classifies by asking a chain of simple yes or no questions about these features ("is this value above that threshold?"); a gradient-boosted tree builds many such trees in sequence, each new one trained to correct the errors left by the ones before it, so that their combined answer becomes progressively more accurate [7]. This kind of model is one of the strongest available for data organised as a table of hand-crafted features. [10]

The outcome of this comparison is itself a result. The gradient-boosted tree, given that one physically-motivated feature, matches CNN's performance almost exactly, and the reason is revealing. On realistic data the two models turn out to lean on the same physics: the signal is so faint and so eroded by missing data that the network cannot find it from the magnitude curve alone—in fact it fails completely that way—and it succeeds only once it is handed the same

single-lens-fit residual the tree relies on, read as a full curve by the network and as a handful of numbers by the tree. The tree is therefore kept not as a competing product but as an independent confirmation that the accuracy reached is the genuine ceiling of what this data allows, and not a weakness of either model in particular. The CNN remains the main model of this work as it still learns, on its own, how to read that residual and combine it with the rest of the curve, and it is the version carried into the web application, while the gradient-boosted tree serves throughout as the tool that verifies its limits.

Data used

Both versions of the model are trained and tested entirely on synthetic data produced by the generator described in further sections, not on real survey observations. This is a deliberate and necessary choice. Supervised learning requires every training example to carry a known label, and for a real microlensing event the true answer (single or binary) is exactly what is unknown and hard to establish; confirmed planetary events, in particular, are rare. The synthetic generator, by contrast, produces as many light curves as needed and knows the nature of each one by construction, since it is the generator itself that decides whether a curve is single or binary before drawing it. This provides the large, perfectly labelled dataset that training demands.

The two versions of the model are trained on two different datasets, which differ only in how realistic they are. The Simple version uses a dataset of 100000 perfect light curves (curves converted to I-band magnitude but left free of photometric noise, cadence gaps and blending). The Real version uses a larger dataset of 250000 light curves to which the full set of OGLE-IV imperfections described earlier (photometric noise, cadence gaps and blending) has been applied. The Real dataset is deliberately the larger of the two because its task is far harder: the features that distinguish the two classes are partly buried under noise and missing data, and more examples give the model a better chance of learning what little signal survives.

Each light curve, in both datasets, has exactly the same form: a sequence of 400 brightness values sampled on a fixed, dimensionless time axis $\tau = (t - t_0)/t_E$ running from −3 to +3, together with a label marking it as single-lens or binary-lens. Sampling every curve on this common grid means the events are registered aligned so that the moment of maximum magnification always falls at the center of the curve and time is measured in units of the event's own duration t_E. This makes every curve directly comparable and gives the model a well-defined, fixed-length input. On the synthetic data this alignment is exact, because t_0 and

t_E are known; it is worth noting that applying the model to genuine survey data would first require estimating these two quantities in order to place each real event on the same grid. In the Real dataset, the points that the observing cadence would have missed are left empty rather than invented, so on average 78% of the 400 values of a curve are missing, a direct reflection of how sparsely a real survey samples an event.

In both datasets 15% of the events are binary and 85% are single, an imbalance ratio of roughly 5.7 to 1. This fraction is not the true astrophysical rate (genuine planetary events are far rarer) but a practical choice: it supplies enough binary examples for the model to learn from while keeping the majority class dominant, closer to the situation a real classifier faces. The consequences of this imbalance, and the steps taken so that it does not simply push the model to label everything as single, are addressed in the Training section.

Following the standard practice introduced earlier, each dataset is divided into three disjoint parts, drawn so that the 15 % binary proportion is preserved in each: a training set (about 72%), used to fit the model; a validation set (about 13%), used to monitor the model during training and to choose its settings; and a held-out test set (15%), which the model does not see at any point during training and which is used only once, at the end, to measure its true performance on unseen data. For the Real dataset this corresponds to 180625 training, 31875 validation and 37500 test curves; for the Simple dataset, 72250, 12750 and 15000. The split is fixed by a random seed so that the experiment is exactly reproducible.

Finally, it is worth stressing that the Real CNN and the gradient-boosted tree of the previous section are trained on the very same 250000 curves. They differ not in the data they see but in the form in which that data is presented to them (the raw curve for the network, a table of engineered features for the tree) which is the subject of the next section.

Input representation

A model never sees a light curve as a person would; it sees an array of numbers. How those numbers are arranged before they reach the model, the representation, is often as important as the model itself, and it is where most of the work of this section lies. The starting point in every case is the same raw curve of 400 magnitude values described above, but each model is handed a different, carefully prepared view of it.

Channels

The convolutional network does not read a single sequence of 400 numbers but several parallel sequences, all aligned point by point, called channels. The idea is the same as in a colour photograph, which is not one image but three superimposed layers (red, green and blue) holding a separate value at every pixel; the network reads all three together. Here, in the same way, each of the 400 time-points can carry several values at once such as its brightness, whether it was actually observed, how symmetric it is, and so on, stacked as channels. At every position along the curve the network sees the whole stack, so it can combine these different views when deciding what a given stretch of the curve means. Choosing what each channel should contain is how the physics of the problem is built into the input.

Simple model

For the Simple model, trained on perfect curves, two channels suffice. The first is simply the magnitude curve itself, centered and scaled by its own spread so that curves of different brightness become comparable. The second channel encodes the single most useful piece of physics for this problem: symmetry. A single-lens (Paczyński) light curve is exactly symmetric in time about its peak because the magnification depends only on $u(\tau) = \sqrt{(u_0^2 + \tau^2)}$, which is unchanged when τ is replaced by $-\tau$ so the brightness a moment before the peak always equals the brightness the same moment after it. A binary lens breaks this symmetry. The second channel therefore holds the fold residual, $R(\tau) = I(\tau) - I(-\tau)$, the difference between each point and its mirror image across the peak. For a genuine single lens this channel is zero everywhere; any departure from zero is the fingerprint of a companion. Both channels are divided by the same scale, the curve's own spread, rather than each being scaled separately, so that the true size of the asymmetry relative to the peak is preserved and a tiny numerical wobble is not blown up into an apparent signal.

Real model

On realistic data the same two ideas remain, but two new difficulties appear (missing points and buried signal) and three further channels are needed to cope with them. The first channel is again the magnitude, scaled as before, with the missing points set to zero. Because a zero would otherwise be indistinguishable from a genuine measurement of that value, the second channel is an observed mask: a simple 1-or-0 track that tells the network which of the 400 points are real measurements and which are cadence gaps. The third and fourth channels carry

the fold residual and its own mask: the same symmetry signal as in the Simple model, but computed only where both a point and its mirror were actually observed, with the fourth channel marking exactly where that was possible.

This is where the central difficulty of the realistic case becomes clear. The fold residual needs a point and its mirror to both survive the observing cadence, and on a curve where roughly 78% of the points are missing, such matched pairs are rare. Therefore the symmetry signal, which was decisive on perfect curves, is largely starved of data. Overcoming this is the purpose of the fifth channel, and of the data treatment it rests on.

The fifth channel (the single-lens-fit residual) and the feature table used by the gradient-boosted tree, both come from the same physically-motivated procedure. The idea is to fit to each curve the best possible pure single-lens model (the smooth, symmetric Paczyński curve that comes closest to the data) and then measure how badly that fit fails at every observed point. The reasoning is simple: a true single lens will be fitted almost perfectly, leaving only noise behind, whereas a binary contains an extra feature that no single-lens curve can reproduce, so the fit is forced to leave that feature behind as a conspicuous residual. Crucially, unlike the fold residual, this one uses every observed point, not only the rare matched pairs, which is exactly why it survives the heavy missingness that cripples the symmetry channel.

In practice the fit is done in terms of flux rather than magnitude, because there the model takes a particularly simple form: the observed flux of a blended single-lens event is $F(\tau) = a \cdot A(u_0, \tau) + b$, where A is the Paczyński magnification, a is the brightness of the lensed source and b the constant light of any unrelated blend. For a fixed shape parameter u_0 this expression is linear in the two brightnesses a and b, so the best pair can be found exactly and instantly; the fit therefore reduces to trying a range of values of u_0 and, for each, solving for the two brightnesses (with the physical requirement that neither brightness can be negative). Each point's leftover is then divided by its expected photometric noise, so that the residual is expressed in units of standard deviations: a deviation only counts as large if it genuinely exceeds the noise at that point. The sum of these squared standardised residuals is the quantity χ^2, a single number measuring how badly the single-lens model fails overall. For the network, this standardised residual is supplied as the fifth channel (gently compressed, so that a handful of very large values cannot dominate the rest); for the tree, the same fit is distilled into numbers.

Feature Engineering for the Gradient-Boosted Tree

The gradient-boosted tree cannot read a curve; it reads a table of features, one row of numbers per event. From the single-lens fit it is given the overall χ^2, the size of the single largest

standardised residual, where along the curve that largest deviation falls, how many points exceed a few standard deviations, and the best-fit shape parameter u_0, among others. These are complemented by a few simple summary statistics of the curve such as how much of it was observed, its overall spread and asymmetry, and the strongest fold residua, so that the tree has both the physically-motivated anomaly measurements and a description of the curve's general shape. This table is the "hand-crafted features" referred to in the Type section: the same underlying signal the network receives, but reduced to a fixed set of numbers rather than presented as full-length sequences the model can scan for itself.

In short, the network and the tree are given the identical underlying signal—how a curve departs from a smooth single lens—in two different forms: the network receives it as a full residual curve it can scan, the tree as a compact set of numbers. That the two arrive at almost the same accuracy from these very different representations is what makes their agreement meaningful.

Architecture

Having established why a convolutional network suits the problem, this section describes the specific network used. It is deliberately small ,around 53000 adjustable parameters, modest by the standards of modern deep learning. This is a considered choice rather than a limitation: the signal to be found is faint and, as later sections show, fundamentally limited in how much it can reveal, so a larger network would gain nothing and would instead be prone to memorising the training data. The same architecture is used for both the Simple and the Real version, so that the two are compared on equal terms; the only difference between them is the number of input channels described earlier (two for Simple, five for Real) which changes only the width of the very first layer (52449 parameters against 53121).

The network is built in two stages: a convolutional trunk that extracts features from the curve, and a small classifier head that turns those features into a decision.

The convolutional trunk is the part of the network that extracts features from the curve, and it is built from three successive blocks. The heart of each block is a set of filters. A single filter is a short pattern—here, just a handful of numbers—that is slid step by step along the curve; at each position it is laid over the few points beneath it, and the network measures how closely that short stretch of curve matches the filter's pattern, producing a single number that is large where the match is good and near zero where it is not. Sweeping the filter across every position turns the original curve into a new sequence, a feature map, that lights up wherever that particular pattern occurs along the curve. A block does this not with one filter but with many at once, each

carrying a different pattern, so it produces many feature maps in parallel, each highlighting a different kind of local shape. Crucially, these patterns are not decided in advance: the numbers that make up each filter are precisely what the network adjusts during training, so it discovers for itself which shapes are worth looking for.

After the filters, each block applies two further operations. The first is batch normalisation. During training the curves pass through the network in batches (groups processed together) and for each filter this operation gathers all the responses that filter produced across the whole batch and standardises them: it subtracts their average and divides by their spread, so the outputs end up centered on zero with a consistent scale, after which two small adjustable numbers let the network rescale and shift the result if that turns out to be useful. The purpose is to cure a specific problem of deep networks. Because the filters in every layer are changing throughout training, the range of the numbers arriving at the next layer keeps drifting, so each layer is perpetually adapting to a moving target, which slows learning down. By forcing each filter's output back onto a stable, predictable range at every step, batch normalisation removes that moving target (the next block always receives its inputs on a familiar scale) which makes training substantially faster and more stable, and far less sensitive to how the network's values happened to be set at the start [14].

The second operation is the ReLU activation. It acts on every individual number coming out of the filters—that is, on each value of each feature map, at each position along the curve (after batch normalisation has rescaled them). Its rule is as simple as possible: if a value is negative it is replaced by zero, and if it is positive it is left unchanged. Recall that a filter's response is large and positive where the curve resembles the pattern it looks for, and negative where the curve resembles its opposite; ReLU therefore discards these "anti-matches" and keeps only positive evidence that a pattern is actually present. Trivial as it looks, this step is what makes stacking blocks worthwhile, because it is non-linear: without an operation of this kind between the layers, the whole network could be algebraically collapsed into a single straight-line transformation, and would be incapable of representing the curved, complex shapes that separate a binary event from a single one.

The blocks are joined by a further operation, applied between them rather than inside every one: pooling, which shortens the curve. Once the filters and the ReLU of a block have produced their feature maps, a max-pooling step walks along each map in small windows (here two points wide) and keeps, from every window, only the single largest value, discarding the other. Each feature map is thereby halved in length: the first block takes the 400-point input down to 200, the second from 200 to 100, while the third is left unpooled. Two things are gained. The plain

one is economy (the deeper blocks work on shorter sequences, so the network stays small and fast). The important one is what is kept: because pooling reports only the strongest response in each window and throws away where within that window it occurred, a feature comes to be remembered as having been found somewhere in a region rather than at one precise point, which makes the detection steadily more indifferent to the exact placement of an anomaly along the curve. That the maximum is taken, rather than an average, is deliberate because, for instance, a caustic crossing is a sharp, narrow spike, and averaging it against its quiet neighbours would wash it out, whereas keeping the maximum lets that one tall value survive the whole reduction from 400 points to 100 intact.

With these operations defined, the specific trunk can be stated exactly. It is three blocks deep. The first block applies 32 filters, each seven points wide; the second, 64 filters five points wide; the third, 128 filters three points wide, so as the network deepens its filters grow more numerous but individually narrower. The narrowing is intentional: the early, wide filters look across a broad stretch of curve and catch gradual features such as the rise and fall of the main peak, while the later, short filters act on feature maps that have already been pooled down, where a span of only a few points now corresponds to a much longer stretch of the original curve and picks out the fine, sharp structure a companion adds. The widening in number (32, then 64, then 128) reflects that there are far more ways to combine simple features into complex ones than there are simple features to begin with, so more filters are needed at each level to hold the growing vocabulary of shapes. After the third block the event has become 128 feature maps, each 100 points long: 128 different learned patterns, each traced along the length of the curve.

This output must now be turned into a single decision, which is the task of the classifier head. It begins by collapsing every feature map to one number through a pair of global pooling operations carried out side by side. Global average pooling takes each of the 128 maps and replaces its 100 values with their average, measuring how strongly its pattern is present across the curve as a whole; global maximum pooling takes the same 128 maps and keeps, from each, only its single largest value, measuring how strongly the pattern shows at its most pronounced point. The two sets of 128 numbers are joined into a single vector of 256, and the pairing is purposeful for the same reason the earlier pools were taken as maxima: the average is sensitive to broad, gentle distortions spread over much of the curve, whereas the maximum is sensitive to a lone sharp spike confined to one place, and a binary event may betray itself through either. Keeping both gives the network the widest chance of noticing a faint anomaly, whatever its shape. This step also fixes, once and for all, the size of what follows (whatever the length of the

curve and wherever along it the telling feature happened to fall, the trunk is reduced to exactly 256 numbers).

Those 256 numbers pass through a small fully-connected network that delivers the verdict. A first layer combines them into 64 intermediate values, a ReLU again discards the negative ones, and a final layer combines those 64 into a single number. Between the two sits dropout: during training a randomly chosen 30 % of the 64 values are set to zero on each pass, a different selection every time, so that no single value can become indispensable and the network is forced to spread its evidence across many features rather than lean on any one. This is a standard and effective guard against a small network simply memorising its training examples [41]. The single number that emerges is the network's raw score; passed through the logistic function, which squeezes any value onto the range 0 to 1, it becomes the model's estimate of the probability that the curve is binary rather than single. The one quantity the whole architecture exists to produce.

Training

With the architecture defined and the data prepared, the model must be trained to adjust its filters and weights. Because the dataset is highly imbalanced (85% single-lens events and 15% binary-lens events) the training procedure requires specific interventions to prevent the network from adopting trivial solutions and to ensure it generalizes to unseen data.

In an 85/15 dataset, a naive model could achieve an 85% accuracy simply by ignoring the light curves entirely and guessing "single-lens" every time. While technically accurate, such a model is scientifically useless. To prevent this, the network is trained using a weighted Binary Cross-Entropy (BCE) loss function. The loss incurred when the model misses a true binary event (a false negative) is multiplied by a positive weight equal to the class imbalance ratio (approximately 5.67). This heavily penalizes the model for ignoring the minority class, forcing it to actively search for the faint anomalies that define a planetary event.

The network's weights are iteratively updated using the Adam optimizer, an adaptive algorithm that dynamically adjusts the step size for each parameter based on past gradients. To ensure the model settles into the best possible configuration, a learning rate scheduler (ReduceLROnPlateau) is employed. As the model trains, it tracks its performance on the validation set; if the performance stops improving for several epochs, the learning rate is automatically reduced. This allows the network to stop taking large, oscillating steps and instead

make the microscopic adjustments needed to crack the fainter, more ambiguous planetary signals.

During training, the model's progress is measured using the Area Under the Receiver Operating Characteristic Curve (AUC) on the validation set, rather than raw accuracy or a fixed-threshold F1 score. AUC is threshold-independent: it measures the model's fundamental ability to rank binary events above single-lens events across all possible cut-off points. To prevent overfitting, the training loop implements Early Stopping. If the validation AUC fails to improve after a set number of epochs, training is halted early and the weights from the best-performing epoch are permanently saved. The model never sees the held-out test set during this process. The final decision threshold (the cut-off point used to declare an event binary or single) is only chosen after training is complete, tailored to maximize precision on the validation set before the final evaluation on the unseen test data.

Results

While the AUC metric measures the model's fundamental ranking ability during training, a deployed astronomical pipeline ultimately requires a hard decision boundary. For this reason, the final models are evaluated using the F1 score. This metric evaluates practical performance at a specific threshold by taking the harmonic mean of Precision (of the events flagged, how many were truly binary?) and Recall (of all the true binaries, how many were successfully found?). A high F1 score proves that a model is not just mathematically sound, but actually useful for generating a reliable candidate catalogue.

Simple CNN

The first model evaluated is the Simple version, trained and tested on the idealised dataset of continuous, noise-free light curves. Evaluated on an unseen test set of 15000 curves, the model achieves an AUC of 0.9934 and an F1 score of 0.9858. As shown in the confusion matrix below, the network successfully identifies over 97% of the planetary anomalies (a recall of 0.972) while maintaining a perfect precision of 1, meaning every single event it flagged as a binary was indeed a binary.

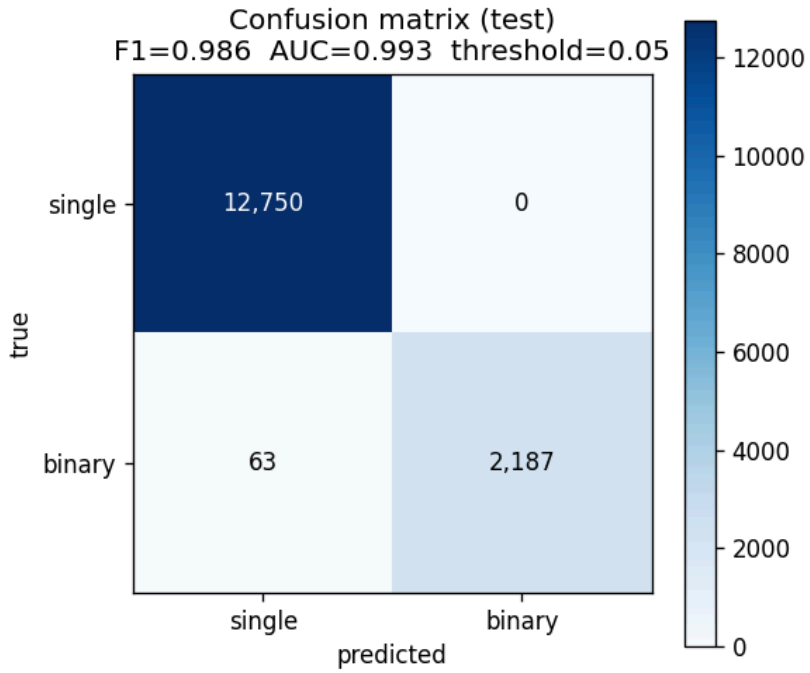


Image 14. Simple version of CNN model's confusion matrix.

These near-perfect metrics establish the theoretical upper bound for this classification task. They confirm that the chosen Convolutional Neural Network architecture is fundamentally sound: when the signal is clean, the network easily isolates the subtle asymmetries caused by a planetary companion.

Real CNN

When applied to realistic data (curves degraded by OGLE-IV photometric noise, blending, and an average of 78% missing observations) the network's performance drops significantly. Evaluated on a test set of 37,500 curves, the Real model achieves an AUC of 0.5995. This drop is not a failure of the architecture but the severe cost of realism.

The observational imperfections physically obscure or destroy the delicate planetary signatures that the Simple model was able to easily detect. Because the data is so degraded, deploying this model on an actual astronomical survey requires a carefully calibrated approach. To balance the need to discover rare planets against the high cost of manually investigating false positives, the continuous probability output of the model is evaluated using a two-stage threshold strategy:

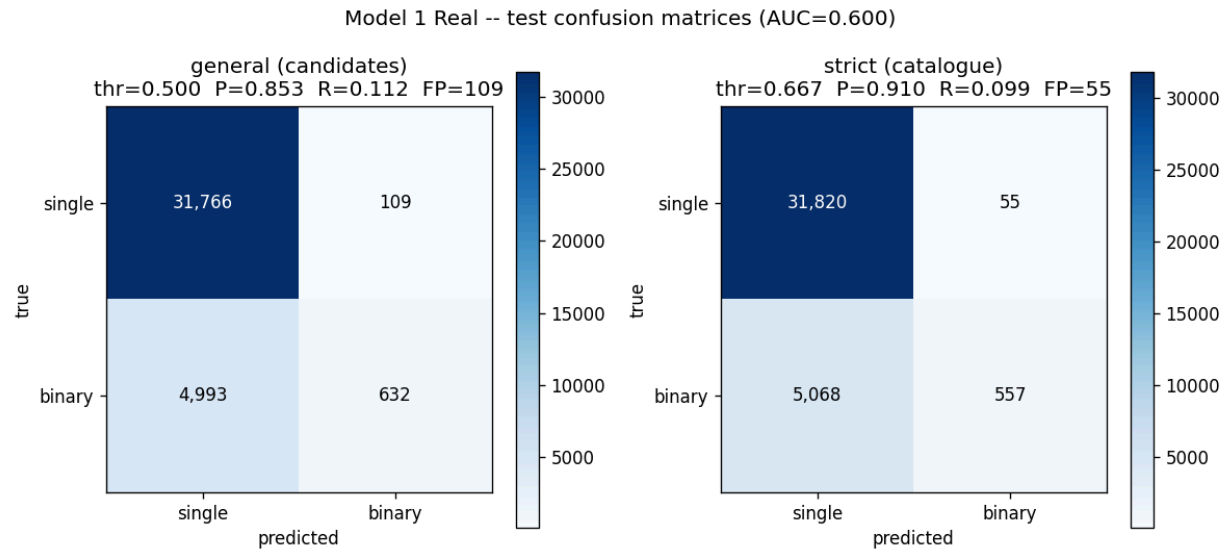


Image 15. Real version of CNN model's confusion matrices showing the general threshold on the left and the strict one on the right.

General Stage: Using a standard probability threshold (0.5), the model acts as a broad net. It prioritizes capturing as many candidates as possible for further review (yielding its highest recall). However, because faint planetary signals are easily mimicked by random noise spikes in the data, this stage inevitably lets in false positives, achieving a precision of roughly 85% and capturing a recall of 11.2% (632 true positives, 109 false positives).

Strict Stage: To produce a highly reliable list of candidates, a strict, calibrated threshold (0.667) is applied. This stage enforces a strict precision requirement: it deliberately sacrifices recall (flagging only the most obvious, high-signal anomalies) to ensure that the events it does flag are highly trustworthy, raising the precision to 91% at a recall of 9.9% (632 true positives, 109 false positives).

In a real survey producing millions of light curves, this two-stage approach allows astronomers to use the General stage to broadly sweep the data, and the Strict stage to generate a prioritized list of high-confidence planetary candidates that are genuinely worth costly follow-up observations.

GBT

As established previously, a Gradient-Boosted Tree (GBT) was constructed as a classical alternative to verify whether the CNN's performance drop was a flaw in the network or a limitation of the data itself. Unlike the CNN, which reads raw sequential data and discovers its own features, the GBT was fed a compact table of explicitly programmed, physically-motivated

parameters; most notably the χ^2 residuals measuring how badly a pure single-lens model failed to fit the light curve.

Evaluated on the exact same realistic test set, the GBT achieves an AUC of 0.6047. This result—an almost perfectly identical performance to CNN's AUC of 0.5995—is the defining conclusion of this analysis. Two radically different algorithms, one relying on spatial pattern recognition and the other on explicitly engineered mathematical physics, converged on the exact same performance ceiling. Therefore, the performance gap between the Simple model and the Real model is not a failure of deep learning.

Instead, an AUC of about 0.60 represents the absolute physical limit of retrievable information contained in actual OGLE-IV survey data. The combination of photometric noise, blending, and severe cadence gaps physically erases the fainter planetary signatures from the light curves, rendering them mathematically undetectable by any computational method.

Web Application & Interactive Tool Hub

The final product of this work includes the several machine learning models developed in this study, integrated into an interactive web application that allows the user to test the models and to generate datasets with specific characteristics.
Rather than presenting static light curve figures in print, the visual exploration and inspection of light curves are intentionally delegated to this interactive platform, allowing users to dynamically examine single-lens and binary-lens events, observe cadence gaps, and inspect caustic anomalies in real time.

Website operation

The web application serves as a graphical interface to make the underlying pipeline accessible and user-friendly.
Scientific programs are usually focused only on producing the final desired product; yet, for the user, obtaining exactly what they want often requires a deep understanding of the project. To make this easier, the web is designed to be intuitive and informative at the same time and, most importantly, does not require the user to write or execute any code but only to follow the intuitive UI and get the desired result as easily as possible.
Every component and process of the web is uploaded to a Git repository available at [35]. To use it, simply clone the repository from GitHub and run the file run.bat. To understand the different aspects of the project and how to use the web properly, it is also necessary to read the README.md file included in the repository.
The repository is actively maintained, so users are encouraged to pull the latest updates to ensure access to the most recent features and corrections.

Datasets generation

The first section of the website is dedicated to generating correct and physically possible datasets according to a set of characteristics that the user must define.
First, the user must select the number of events the dataset will contain. The web accepts between 10 and 500000 events, but it is important to be aware that each event takes some time to compute, so the larger the dataset, the longer it will take to be created.

Next, the user specifies the desired proportion of binary-lens events, from 0 to 100%.

The number of points per event must also be specified. More points mean more information and easier-to-interpret events, but again a longer computation time. The event duration is normalized from $-3t_E$ to $+3t_E$ across the specified number of points.

Note that the models are trained in 400 points datasets therefore will only accept this configuration.

After that, the user selects whether the dataset should be expressed in A(t) (dimensionless amplification) or in I(t) (I-band magnitude).

If the user selects I(t), firstly each event is additionally assigned its own source baseline magnitude I_s, as a magnitude curve simply cannot be produced without knowing how bright the source is. Then the option to add observational imperfections also becomes available. These imperfections (photometric noise, observing cadence and blending) have been extracted from the OGLE-IV survey, and their only purpose is to make the generated events realistic. When they are enabled, each event also receives its own blend fraction f_s.

The light curve is then built from both: the peak is diluted by the constant blend light, Gaussian noise of the appropriate size is added to every point, and the points that would not have been observed are left empty. Note that later in the project, the imperfections of other telescopes—for instance NASA's Roman Space Telescope [24]—could be extracted and added in exactly the same way.

Finally, the user can decide which columns the dataset should contain: all the parameters, only the single-lens ones, none of them and just the light curves, and so on. To make this faster and easier, some presets are available in the top-right corner of the output-columns subsection.

Once everything is selected, the dataset is generated and saved to a temporary cache.

When the dataset is generated, the web displays its distributions and also offers 5 random events from each class which can be rerandomized as a preview of the events. Additionally, users can search for a specific event to examine its light curve in detail. All physical parameters for that event are displayed, even if they were excluded from the final CSV output, because the full event data remains temporarily stored in the cache.

From there, the user can download the dataset with the selected configuration as a CSV or PKL file, or run the built-in validation, whose details will be elucidated in the next section, to confirm the dataset is physically realistic.

Dataset validation

Generating a dataset is not enough: one must also prove that what came out is what was intended. Every parameter of every event is drawn at random from a distribution, and a mistake anywhere in that sampling such as a wrong bound, a misplaced exponent, or a distribution silently truncated, would not crash the program. It would simply produce a dataset that looks perfectly reasonable while describing a universe that does not exist, and any model trained on it would learn the wrong physics without ever complaining. For this reason the website includes a validation tool, which can be run on any dataset generated by the web, or on one uploaded by the user, and which checks parameter by parameter that the data really follows the distributions established in this work.

The comparison is made with the Kolmogorov-Smirnov (KS) test [16], in the two-sample form that compares two sets of data directly against one another [38]. The idea behind it is simple. Instead of comparing two histograms bin by bin (which depends on how the bins happen to be chosen) one compares their cumulative curves: for every possible value, the fraction of events that fall below it. Two identical distributions produce two identical cumulative curves, and the KS statistic is simply the largest vertical gap between them. A value of 0 means a perfect match, and a value of 0.05 means that, at worst, the two distributions disagree about 5 % of the events.

One methodological point deserves emphasis, because it is a trap that is easy to fall into. The test also returns a p-value, and it is tempting to demand that it be large before declaring a match. With datasets of this size, however, that criterion is worse than useless. The p-value answers the question "could this difference have arisen by chance?", and with tens of thousands of events the answer is always no: even a completely negligible discrepancy becomes "statistically significant", and the p-value collapses towards zero for any pair of real distributions. A p-value threshold could therefore never be passed, no matter how good the generator was. What matters is not whether a difference exists, as with enough data, one always does, but whether it is large enough to matter, and that is precisely what the KS statistic measures. Every check is therefore judged on the statistic, requiring it to stay below a small threshold, and never on the p-value.

Each physical parameter is checked against the reference distribution defined earlier in this work: the lens mass, the distances to the lens and to the source, the lens velocity, the impact parameter, the source baseline magnitude, and, for binary events, the mass ratio, the semi-major axis, the orbital eccentricity and the trajectory angle. For every one of them the tool reports the reference used, the value measured in the generated data, the criterion applied, and

whether it passed. It also produces the corresponding figures, in which the generated histogram is drawn on top of the reference curve, so that any disagreement is immediately visible rather than hidden behind a number.

When the OGLE imperfections are enabled, four further checks are added, one for each ingredient of the realism described earlier:

- Noise level. The distribution of the photometric uncertainties actually applied to the generated curves is compared with the one measured from real OGLE-IV photometry. Only near-baseline points are used, so that the comparison is made like with like: the synthetic curves deliberately sample the magnified peak densely, which real observations do not.
- Cadence coverage. The fraction of points of each light curve that survived the observing schedule is measured, confirming that the gaps are genuinely present and that no curve remains fully sampled.
- Blend fraction. The distribution of f_s applied to the dataset is compared with the OGLE-IV catalogue from which it was drawn.
- Observed baseline. The baseline magnitude implied by each generated event, $I_{base} = I_s + 2.5 \cdot \log f_s$, is compared with the baselines measured directly from the raw photometry.

This last check is of a different nature from all the others, and it is by far the most valuable. Every other test compares the generated data against the very distribution it was drawn from; passing them proves that the sampling is faithful, but not that the model is right, as a perfect reproduction of a wrong assumption would pass just as easily. The baseline check, by contrast, compares a prediction against data that was never used to build the model. The values of I_s and f_s come from the fitted event catalogue, while the baselines they are compared against are measured from the raw photometry: an entirely separate product of the survey, obtained in a different way. Nothing forces the two to agree. That they do (the generated baselines reproducing the observed ones to within a few hundredths of a magnitude, and across the whole distribution rather than merely at its center) is genuine, independent evidence that the blending model describes the real survey and not merely itself.

Run on a representative dataset, all fourteen checks are passed. The source magnitude matches its reference with a KS statistic of 0.021, the blend fraction with 0.032 and the noise level with 0.045; roughly half of the points of each light curve are removed by the observing cadence; and the observed baseline is reproduced with an offset of only 0.08 mag. The generated datasets can therefore be considered not merely physically plausible, but

demonstrably consistent with the survey they are built to imitate — which is the condition any of them must satisfy before a model is trained on it.

Distribution logic

All the distributions used to validate the datasets, which are the same explained on Parameters distributions in this work, are then shown and briefly explained in the easiest way possible to, again, make the web more user-friendly

Models

Finally, the last section of the web application contains the models inference interfaces. First of all, regarding the simple model, a brief description of it and its training process is explained. The required format of the input dataset is also explicitly stated. As it is the simple model, a dataset containing cadence gaps will be automatically rejected as well as any dataset where the lightcurves are represented by any number different than 400.

The user has the option to classify the dataset currently in cache or to upload a custom one. Providing access to the Simple model allows users to independently verify the theoretical maximum detection limits explained in this study.

After this a brief explanation of the real model, the CNN, and the reason for having two different filters at different thresholds is elucidated. Then an honest expectation of what the results could be is also displayed together with one of the conditions before upload that the simple model already had which is that the events on the dataset must be exactly 400 points long.

Both options, to feed a dataset in cache and to upload one, are possible which allows the user to feed actual real data from real microlensing surveys to the model and get its predictions. The predictions always include the probability of being binary yet the option of not including it on the resulting dataset is also available.

Note that the web application deploys only the CNN models for interactive inference; the GBT was retained as an offline tabular cross-check to independently corroborate the physical information ceiling of data conditions.

Finally, when the dataset has been evaluated by the model, the user gets the option to download the same dataset that was uploaded with the added column containing the model prediction for each event and, if desired, the binary probability. There is also available the option to only download the events that have been selected as binary by the model.

Real data

After training and evaluating the models entirely on synthetic data designed to mimic reality, the pipeline must finally be tested on genuine observational data to verify its practical relevance.
As has been explained on previous sections, the real CNN model does not get a high number of real positives from the generated dataset (for instance, on the test set during the training the strict threshold only flagged about 9% of the real events and 1% false positives) yet real surveys' models don't get 100% precision either. The final objective of this section is to understand how many of the findable events, the ones found by OGLE, can also be found with our CNN model.

Data preprocessing

Until this point, the models have been trained and evaluated entirely on synthetic data. While the synthetic generator was meticulously designed to reproduce OGLE-IV observational imperfections, it remains a simulation. To definitively prove the pipeline's physical relevance, the model must be tested against genuine observational data where the ground-truth is known. To achieve this, data from real, confirmed, planetary microlensing events was extracted.
The NASA Exoplanet Archive was queried for all confirmed exoplanets discovered via microlensing. This list was then filtered to include only events observed by the OGLE-IV survey from 2011 onwards, allowing the target events to be cross-referenced with the public OGLE Early Warning System (EWS) archive. This process yielded a final target list of 92 unique, verifiable planetary events. Because every curve in this dataset hosts a confirmed planet, the ground-truth label for all 92 events is known to be binary-lens.
To feed these real observations into the Convolutional Neural Network, the raw photometry had to be reformatted to match the model's fixed 400-point input representation. The raw EWS data contains entire observing seasons, whereas the model expects a focused window spanning six Einstein times. Therefore, each event was windowed using its officially published single-lens fit parameters (t_0 and t_E). The raw observational timestamps (HJD) were converted to the dimensionless event time τ, and all photometric measurements falling within the target window were binned into the 400 uniform slots. Time slots with no corresponding measurements were left empty to faithfully represent the cadence gaps.

Through this preprocessing, the raw telescope telemetry was transformed into the exact same standardized format as the synthetic training data, allowing the model to analyze the genuine curves without any structural modifications.

Results

When the 92 real, confirmed planetary events were processed by the trained Real CNN, the model successfully flagged 53.3% of the planets at the general threshold, and 47.8% at the strict threshold. At first glance, catching roughly half of the planets might appear low when compared to the near-perfect detection rate of the Simple model. However, placed in the proper astronomical context, this result represents a massive success for the pipeline.

When the Real model was evaluated on the synthetic test set, its recall was 11.2% at the general threshold and 9.9% at the strict threshold. This is because the synthetic dataset simulates the true, unbiased universe, where the vast majority of planetary anomalies are so minuscule that they are completely swallowed by the telescope's photometric noise. They are physically undetectable by this method. The 92 genuine observational events, by contrast, represent a heavily biased sample: they are the planets that were actually discovered. Their planetary signatures were strong enough to survive the noise and be noticed by professional astronomers and advanced computational pipelines. The fact that a 1D Convolutional Neural Network trained entirely on synthetic, computer-generated data was able to successfully identify roughly half of the confirmed planets discovered by humanity in the OGLE-IV survey over the last 15 years, purely by scanning the raw photometric shape without any manual feature engineering, is a remarkable validation of the project. It definitively proves that the synthetic dataset generator is an incredibly faithful simulation of reality, and that the resulting machine learning pipeline successfully bridges the gap between theoretical physics and actual astronomical observation.

Conclusions

Gravitational microlensing is, by nature, a difficult field to approach. It sits at the intersection of general relativity, stellar astrophysics, and observational astronomy. Disciplines that individually demand years of formal study. The available literature is sparse, highly specialized, and rarely written with accessibility in mind. There are no standardized public datasets for machine learning experimentation, no beginner-friendly textbooks on the binary-lens equation, and no pre-built tools for generating realistic synthetic light curves. Every component of this project—from the physics engine to the noise models to the neural network—had to be understood and constructed from the ground up.

The scope and resource constraints of this work must also be acknowledged. This project was developed as a research project by a single student, without access to university-level computational infrastructure, professional astrophysics supervision, or proprietary survey data. Every dataset was generated on personal hardware. Every empirical model (the photometric noise, the cadence gaps, the blending) was derived exclusively from publicly available OGLE-IV data. The machine learning models were trained, debugged, and iterated upon without prior formal education in deep learning. The decision to combine two inherently complex fields such as computational physics and artificial intelligence into a single high school project was ambitious, and the path was rarely straightforward.

Building functional machine learning models with minimal programming experience was the single most demanding challenge of the entire process. The development relied heavily on AI-assisted coding tools, which introduced their own layer of complexity: understanding, debugging, and validating code that was not written entirely by hand required developing a different kind of technical literacy—the ability to critically evaluate generated solutions, identify errors in logic that a language model cannot catch, and maintain full scientific ownership over every architectural and methodological decision.

Despite these constraints, the results achieved speak for themselves. A physically rigorous synthetic dataset generator was built and validated through 14 independent statistical checks, including a cross-check between two entirely independent OGLE-IV data products. Two convolutional neural networks and one gradient-boosted tree were trained, and their independent convergence at an AUC of approximately 0.60 allowed this work to quantify the physical information ceiling imposed by OGLE-IV survey conditions—a result that is not a limitation of the models, but a measured property of the data itself. Most remarkably, when the

trained CNN was evaluated against 92 real, confirmed planetary microlensing events discovered by the OGLE-IV survey over the last 15 years, it successfully flagged 53.3% of them. A model trained entirely on synthetic, computer-generated data was able to detect real planets in real telescope observations. The entire pipeline (from dataset generation to model inference) was then packaged into an interactive web application, making the tools accessible beyond the scope of this paper.

This project was born from the desire to bridge two disciplines that rarely meet at this academic level: theoretical physics and applied machine learning. It has confirmed that meaningful scientific computation is not exclusive to university laboratories or professional research teams. The tools, models, and methodology developed here are publicly available, and the author intends to continue exploring the intersection of physics and data science at university, where the foundations built in this work can serve as a launching point for deeper, more specialized research.

References

1. BREIMAN, Leo. Random Forests. Machine Learning. Online. October 2001. Vol. 45, no. 1, p. 5–32. [Accessed 13 July 2026]. Available from: https://link.springer.com/article/10.1023/A:1010933404324
2. CHAO, Wei-Lun. Machine Learning Tutorial. National Taiwan University. December 2011. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/machine-learning-tutorial%20BY%20Wei-Lun%20Chao.pdf
3. CORTES, Corinna; VAPNIK, Vladimir. Support-Vector Networks. Machine Learning. Online. September 1995. Vol. 20, no. 3, p. 273–297. [Accessed 13 July 2026]. Available from: https://link.springer.com/article/10.1007/BF00994018
4. COVER, Thomas M.; HART, Peter E. Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory. Online. January 1967. Vol. 13, no. 1, p. 21–27. [Accessed 13 July 2026]. Available from: https://ieeexplore.ieee.org/document/1053964
5. DOMINGOS, Pedro. A Few Useful Things to Know about Machine Learning. Communications of the ACM. 2012. Vol. 55, no. 10, p. 78–87. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/a-few-useful-things-to-know-about-machine-learning%20BY%20Pedro%20Domingos.pdf
6. EINSTEIN, Albert. Lens-Like Action of a Star by the Deviation of Light in the Gravitational Field. *Science*. Online. 4 December 1936. Vol. 84, no. 2188, p. 506–507. Online. [Accessed 13 January 2026]. Available from: https://doi.org/10.1126/science.84.2188.506
7. Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 29(5), 1189–1232. Online. [Accessed 27 July 2026]. Available from: https://www.researchgate.net/publication/2424824_Greedy_Function_Approximation_A_Gradient_Boosting_Machine
8. GODINES, Daniel; BACHELET, Etienne; NARAYAN, Gautham; STREET, Rachel A. A machine learning classifier for microlensing in wide-field surveys. Astronomy and Computing. Online. July 2019. Vol. 28, p. 100298. [Accessed 13 July 2026]. Available from: https://arxiv.org/pdf/2004.14347

9. GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. Deep Learning. MIT Press. 2016. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/deep-learning-adaptive-computation-and-machine-learning-series%20BY%20Ian%20Goodfellow.pdf
10. GRINSZTAJN, Léo; OYALLON, Edouard; VAROQUAUX, Gaël. Why do tree-based models still outperform deep learning on tabular data? 17 July 2022. arXiv preprint. Online. [Accessed 25 April 2026]. Available from: https://arxiv.org/abs/2207.08815
11. HAN, Cheongho; CHUN, Mun-Suk; CHANG, Kyongae. Astrometric Properties of Gravitational Binary-Microlens Events and Their Applications. *The Astrophysical Journal*. Online. 20 November 1999. Vol. 526, no. 1, p. 405–410. Online. [Accessed 5 February 2026]. Available from: https://iopscience.iop.org/article/10.1086/307996/pdf
12. HOCHREITER, Sepp; SCHMIDHUBER, Jürgen. Long Short-Term Memory. Neural Computation. Online. 15 November 1997. Vol. 9, no. 8, p. 1735–1780. [Accessed 13 July 2026]. Available from: https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory
13. HOWELL, Steve B. Handbook of CCD Astronomy. 2nd ed. Cambridge: Cambridge University Press, 2006. Online. [Accessed 13 July 2026]. Available from: https://www.researchgate.net/profile/Ali_Tavackolimehr/publication/332392902_Handbook_of_CCD_Astronomy/links/5cb134f6299bf1209762473d/Handbook-of-CCD-Astronomy.pdf?__cf_chl_tk=SpabmgkJ_TLuoqFW9lmv067NBCZcH9wiNTmfJnyEC08-1783935865-1.0.1.1-N_5ajtTv3ZQkEyMUHHDiM1qJob73ku8jnLwSV1aCp.4
14. Ioffe, S. & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. Online. [Accessed 27 July 2026]. Available from: https://arxiv.org/pdf/1502.03167
15. ISMAIL FAWAZ, Hassan; FORESTIER, Germain; WEBER, Jonathan; IDOUMGHAR, Lhassane; MULLER, Pierre-Alain. Deep learning for time series classification: a review. Data Mining and Knowledge Discovery. Online. July 2019. Vol. 33, no. 4, p. 917–963. [Accessed 13 July 2026]. Available from: https://link.springer.com/article/10.1007/s10618-019-00619-1
16. KOLMOGOROV, Andrey. Sulla determinazione empirica di una legge di distribuzione. Giornale dell'Istituto Italiano degli Attuari. 1933. Vol. 4, p. 83–91.
17. LECUN, Yann; BENGIO, Yoshua. Convolutional Networks for Images, Speech, and Time-Series. In: ARBIB, Michael A. (ed.). The Handbook of Brain Theory and Neural Networks. Cambridge, MA: MIT Press, 1995. Online. [Accessed 13 July 2026]. Available

from: https://www.theswissbay.ch/pdf/Gentoomen%20Library/Artificial%20Intelligence/Neural%20networks/The%20Handbook%20Of%20Brain%20Theory%20And%20Neural%20Networks%202Nd%20Ed%20-%20Michael%20A%20Arbib.pdf

18. LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. Deep Learning. Nature. Online. 28 May 2015. Vol. 521, no. 7553, p. 436–444. [Accessed 13 July 2026]. Available from: https://www.nature.com/articles/nature14539
19. LECUN, Yann; BOTTOU, Léon; BENGIO, Yoshua; HAFFNER, Patrick. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE. Online. November 1998. Vol. 86, no. 11, p. 2278–2324. [Accessed 13 July 2026]. Available from: https://ieeexplore.ieee.org/document/726791
20. LI, Qianxiao. Foundations of Machine Learning. National University of Singapore. 2020. Online. [Accessed 4 April 2026]. Available from: https://bpb-us-w2.wpmucdn.com/blog.nus.edu.sg/dist/5/11890/files/2021/10/LectureNotes_DSA5102_2021.pdf
21. MAO, Shude. Introduction to Gravitational Microlensing Online. 2008. Online. [Accessed 12 January 2026]. Available from: https://arxiv.org/pdf/0811.0441
22. MEHTA, Pankaj; WANG, Ching-Hao; DAY, Alexandre G. R.; RICHARDSON, Clint; BUKOV, Marin; FISHER, Charles K.; SCHWAB, David. A high-bias, low-variance introduction to machine learning for physicists. ArXiv e-prints. 29 May 2019. Online. [Accessed 04 April 2026]. Available from: https://arxiv.org/pdf/1803.08823
23. NAGESH, Ajay; RAMAKRISHNAN, Ganesh. CS725 : Foundations of Machine learning - Lecture Notes. Indian Institute of Technology, Bombay. Autumn 2011. Online. [Accessed 4 April 2026]. Available from: https://www.cse.iitb.ac.in/~cs725/notes/classNotes/extra_lecturenotes_cs725.pdf
24. NASA Completes Nancy Grace Roman Space Telescope Construction - NASA. *NASA*. Online. 4 December 2025. Online. [Accessed 13 January 2026]. Available from: https://www.nasa.gov/missions/roman-space-telescope/nasa-completes-nancy-grace-roman-space-telescope-construction/
25. NASA EXOPLANET SCIENCE INSTITUTE. Microlensing. Online. [Accessed 17 February 2026]. Available from: https://exoplanetarchive.ipac.caltech.edu/cgi-bin/TblView/nph-tblView?app=ExoTbls&config=ML&constraint=ml_modeldef=1

26. NASA EXOPLANET SCIENCE INSTITUTE. Planetary Systems. Online. [Accessed 02 March 2026]. Available from: https://exoplanetarchive.ipac.caltech.edu/cgi-bin/TblView/nph-tblView?app=ExoTbls&config=PS
27. NASA Science. Exoplanets [online]. Washington, D.C.: National Aeronautics and Space Administration, 2026. Online [Accessed 27 January 2026]. Available from: https://science.nasa.gov/exoplanets/
28. NILSSON, Nils J. Introduction to Machine Learning. Stanford University. 3 November 1998. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/introduction-to-machine-learning%20BY%20Nils%20J.%20Nilsson.pdf
29. OGLE. OGLE-IV Early Warning System. Warsaw University Astronomical Observatory. Online. [Accessed 13 July 2026]. Available from: https://ogle.astrouw.edu.pl/ogle4/ews/ews.html
30. OGLE IV. Best-fitting parameters of the analyzed microlensing events in low-cadence OGLE fields. Online. [Accessed 17 February 2026]. Available from: https://www.astrouw.edu.pl/ogle/ogle4/microlensing_maps/table3.dat
31. PACZYŃSKI, Bohdan. GRAVITATIONAL MICROLENSING BY THE GALACTIC HALO. *IP The Astrophysical Journal*. Online. 1986. Vol. 304, p. 1–5. Online [Accessed 5 February 2026]. Available from: https://articles.adsabs.harvard.edu/pdf/1986ApJ...304....1P
32. PACZYŃSKI, Bohdan. Gravitational microlensing of the Galactic bulge stars. The Astrophysical Journal. Online. April 1991. Vol. 371, p. L63. Online. [Accessed 17 February 2026]. Available from: https://articles.adsabs.harvard.edu/pdf/1991ApJ...371L..63P
33. POGSON, N. R. Magnitudes of Thirty-six of the Minor Planets. *Monthly Notices of the Royal Astronomical Society, 17, 12–15*. Online. [Accessed 27 June 2026]. Available from: https://articles.adsabs.harvard.edu/pdf/1856MNRAS..17...12P
34. RAHVAR, Sohrab. Gravitational microlensing I: A unique astrophysical tool. *International Journal of Modern Physics D*. 27 May 2015. Vol. 24, no. 07, p. 1530020. Online. [Accessed 13 January 2026]. Available from: https://arxiv.org/pdf/1503.04271
35. RUBIÓ, Roc. Gravitational Microlensing Tool Hub. Source code repository. 2026. Available from: https://github.com/rocrubio/Microlensing-Tool-Hub

36. SHALEV-SHWARTZ, Shai; BEN-DAVID, Shai. Understanding Machine Learning: From Theory to Algorithms. Cambridge University Press. 2014. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/understanding-machine-learning-theory-algorithms%20BY%20Shai%20Shalev-Shwartz%20and%20Shai%20Ben-David.pdf
37. SIMEONE, Osvaldo. A Brief Introduction to Machine Learning for Engineers. King's College London. 2018. Vol. XX, no. XX, p. 1–231. Online. [Accessed 4 April 2026]. Available from: https://arxiv.org/pdf/1709.02840.pdf
38. SMIRNOV, Nikolai. Table for Estimating the Goodness of Fit of Empirical Distributions. The Annals of Mathematical Statistics. June 1948. Vol. 19, no. 2, p. 279–281. Online. [Accessed 13 July 2026]. Available from: https://projecteuclid.org/euclid.aoms/1177730256
39. SMOLA, Alex; VISHWANATHAN, S.V.N. Introduction to Machine Learning. Cambridge University Press. 2008. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/introduction-to-machine-learning%20BY%20Alex%20Smola%20and%20S.V.N.%20Vishwanathan.pdf
40. SOLOMON, Justin. Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics. Taylor & Francis. 2015. Online. [Accessed 4 April 2026]. Available from: https://github.com/ec2ainun/books-ML-and-DL/blob/master/numerical-algorithms%20BY%20Justin%20Solomon.pdf
41. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15(1), 1929–1958. Online. [Accessed 27 July 2026]. Available from: https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf
42. TSAPRAS, Yiannis. Microlensing Searches for Exoplanets. *Geosciences*. Online. 29 September 2018. Vol. 8, no. 10, p. 365. Online. [Accessed 27 January 2026]. Available from: https://www.mdpi.com/2076-3263/8/10/365
43. UDALSKI, Andrzej; SZYMAŃSKI, Michał K.; SZYMAŃSKI, Grzegorz. OGLE-IV: Fourth Phase of the Optical Gravitational Lensing Experiment. Acta Astronomica. Online. 2015. Vol. 65, no. 1, p. 1–38. [Accessed 13 July 2026]. Available from: https://acta.astrouw.edu.pl/Vol65/n1/pdf/pap_65_1_1.pdf

44. VAN EYLEN, Vincent; ALBRECHT, Simon; HUANG, Xu; MACDONALD, Mariah G; DAWSON, Rebekah I; MAXWELL XU CAI; PRICE-WHELAN, Adrian M; LUNDKVIST, Mia S; V. SILVA AGUIRRE; SNELLEN, Ignas; WINN, Joshua N. The Orbital Eccentricity of Small Planet Systems. 2 July 2018. Vol. 157, no. 2, p. 61–61. Online. [Accessed 02 March 2026]. Available from: https://iopscience.iop.org/article/10.3847/1538-3881/aaf22f/pdf
45. WITT, Hans J.; MAO, Shude. On the Minimum Magnification between Caustic Crossings for Microlensing by Binary and Multiple Stars. *The Astrophysical Journal*. 1995. Vol. 447, p. L105–L108. Online. [Accessed 16 October 2026]. Available from: https://articles.adsabs.harvard.edu/pdf/1995ApJ...447L.105W